



УНИВЕРЗИТЕТ У НИШУ
ЕЛЕКТРОНСКИ ФАКУЛТЕТ

**Развој система за подршку раду у апотеци применом
Retrieval-Augmented Generation система**

Завршни рад
Студијски програм: електротехника и рачунарство
Модул: рачунарство и информатика

Студент:

Јордан Динић, бр. инд. 18625

Ментор:

Доц др Милена Б. Фртунић Глигоријевић

Ниш, <месец> 2026. година

Универзитет у Нишу
Електронски Факултет

Развој система за подршку раду у апотеци применом Retrieval-Augmented Generation система

Development of a Pharmacy Support System Using Retrieval-Augmented Generation

Завршни рад
Студијски програм: електротехника и рачунарство
Модул: рачунарство и информатика

Студент: Јордан Динић, бр. инд. 18625

Ментор: Доц др Милена Б. Фртунић Глигоријевић

Задатак: Упознати се са појмом и основним концептима Retrieval-Augmented Generation (RAG) система. Упознати се са основним концептима великих мултијезичних модела за векторизацију текста. Истражити основне карактеристике ChromaDB векторске базе података. У практичном делу рада, коришћењем ChromaDB базе података и великих језичких модела имплементирати систем за подршку раду у апотеци применом Retrieval-Augmented Generation sistema.

Датум пријаве рада: _____
Датум предаје рада: _____
Датум одбране рада: _____

Комисија за оцену и одбрану:

1. Доц др Милена Б. Фртунић Глигоријевић, Председник
Комисије

2. <Prof. / Doc dr Ime Prezime>, Član

3. <Prof. / Doc dr Ime Prezime>, Član

Развој система за подршку раду у апотеци применом Retrieval-Augmented Generation система

САЖЕТАК

У раду је развијен и анализиран прототип система за подршку раду у апотеци заснован на генерисању уз помоћ претраживања, односно Retrieval-Augmented Generation приступу. Намена система је проналажење релевантних делова документације и састављање одговора на питање корисника, уз приказ пронађених извора. База знања обухвата PDF упутства за лекове, водич добре апотекарске праксе и документ о интеракцијама лекова.

Реализовано решење раздваја читавање страница, препознавање структуре докумената, сегментацију, складиштење и одговарање на питања. За представљање текста и метаподатака користи се *LangChain Document*. Упутства за лекове деле се према шест главних секција, водич према расположивим PDF обележивачима, а документ о интеракцијама према посебним правилима за наслове. Имплементиран је *parent-child* приступ: *parent* сегменти служе за дељење PDF докумената на секције, док *child* сегменти служе за претрагу и формирање контекста у промпта. Назив лека, активна супстанца, назив PDF-а и секција укључују се у садржај сегмената када су одговарајући подаци доступни.

Chroma Cloud обавља складиштење и претрагу уз густе *Qwen* и ретке *SPLADE* векторске репрезентације. Рангови се комбинују методом *Reciprocal Rank Fusion*. Одговор генерише *Gemini*, а *Streamlit* интерфејс приказује одговоре, изворе и пронађени контекст. Историја тих података чува се током корисничке сесије, без прослеђивања претходног разговора моделу.

Локални корпус садржи 31 PDF са 339 страница, из којих настају 493 родитељске секције и 1.361 мањи сегмент. У почетној евалуацији са 15 питања, подразумевана хибридна конфигурација постиже макро *Recall@5* од 0,8667 и *MRR@5* од 0,8333, при чему се релевантност оцењује углавном према очекиваном документу и секцији. Проверен је пролазак 31 софтверског теста. Анализа издваја ограничења у раду са ћиричним називом лека, издвајању садржајних секција водича, преношењу доказа у ограничени контекст и прецизности навођења извора. Резултати потврђују функционалност прототипа, али не представљају клиничку валидацију. Систем је намењен подршци стручној процени, а не њеној замени.

Кључне речи: *RAG*, подршка раду у апотеци, мултијезичка векторизација, *ChromaDB*, хијерархијска сегментација, евалуација претраге.

Development of a Pharmacy Support System Using Retrieval-Augmented Generation

ABSTRACT

This thesis presents the development and analysis of a Retrieval-Augmented Generation prototype for pharmacy support. The system retrieves relevant passages from a document collection and generates an answer to the user's question while displaying retrieved sources. Its knowledge base consists of PDF medicine leaflets, a good pharmacy practice guide, and a reference document on drug interactions.

The implemented solution separates page loading, document structure recognition, segmentation, storage, and question answering. The LangChain Document object is used to represent text and metadata. Medicine leaflets are divided according to six main sections, the guide is segmented according to the available PDF bookmarks, and the drug-interaction document is processed according to dedicated title-detection rules. A parent-child approach is implemented: parent segments are used to divide PDF documents into sections, while child segments are used for retrieval and for forming the context included in the prompt. The medicine name, active substance, PDF filename, and section are included in the segment content whenever the corresponding data are available.

Chroma Cloud stores the records and supports search using dense Qwen and sparse SPLADE representations combined through Reciprocal Rank Fusion. Gemini generates the final answer. A Streamlit interface displays answers, retrieved sources, and context, retaining this information within the current session. Previous conversation turns are not supplied to the language model.

The local collection contains 31 PDFs with 339 pages, producing 493 parent sections and 1,361 child chunks. An initial evaluation with 15 questions yielded a macro Recall@5 of 0.8667 and MRR@5 of 0.8333 for the default hybrid configuration. Relevance was primarily assessed against expected document and section metadata. All 31 software tests passed. Identified limitations include a failed Cyrillic medicine-name query, insufficiently informative guide sections, evidence loss under the context budget, and incomplete source attribution. These results demonstrate a functional prototype rather than clinical validity. The system is intended to assist, not replace, the professional judgment of a pharmacist or physician.

Keywords: Retrieval-Augmented Generation, pharmacy support, multilingual embeddings, ChromaDB, hierarchical chunking, retrieval evaluation.

СПИСАК КОРИШЋЕНИХ СКРАЋЕНИЦА И ОЗНАКА

Скраћеница	Значење
API	Application Programming Interface, програмски интерфејс
CLI	Command-Line Interface, интерфејс командне линије
LLM	Large Language Model, велики језички модел
NLP	Natural Language Processing, обрада природног језика
OCR	Optical Character Recognition, оптичко препознавање знакова
PIL / PL	Patient Information Leaflet / Package Leaflet, упутство за пацијента
RAG	Retrieval-Augmented Generation, генерисање уз помоћ претраживања
RRF	Reciprocal Rank Fusion, комбиновање реципрочних рангова
SDK	Software Development Kit, скуп алата за развој софтвера
SPLADE	Sparse Lexical and Expansion Model
UI	User Interface, кориснички интерфејс

САДРЖАЈ

1. Увод.....	8
1.1. Предмет и мотивација	8
1.2. Дефиниција проблема	8
1.3. Циљ и обухват рад	9
1.4. Границе примене система	9
1.5. Организација рада	9
2. Преглед области и постојећих приступа	10
2.1. Претрага документације и генерисање одговора	10
2.2. Развој неуронског проналажења информација	10
2.3. Ретке репрезентације и комбиновање рангова	10
2.4. Избор приступа за практични систем	11
3. Теоријске основе	12
3.1. Структура RAG система	12
3.2. Векторизација и семантичка сличност	13
3.3. Мултијезична претрага и коришћени модели	13
3.4. Векторске базе и ChromaDB	14
3.5. Хибридна претрага и груписање	14
3.6. Сегментација и хијерархијски контекст	15
3.7. Велики језички модел и генерисање	16
4. Анализа проблема и пројектовање система	17
4.1. Корисници, задаци и границе примене	17
4.2. Структура и порекло корпуса	17
4.3. Функционални и нефункционални захтеви	18
4.4. Архитектура и границе одговорности	18
4.5. Модел података и следљивост	19
4.6. Пројектне одлуке и алтернативе	20
5. Имплементација система	22
5.1. Развојно окружење и организација кода	22
5.2. Учитавање PDF докумената	23
5.3. Препознавање секција	23
5.4. Parent-child сегментација	24
5.5. Припрема записа за Chroma Cloud	25
5.6. Векторизација, колекција и упис	25
5.7. Претрага и формирање retrieval излаза	26
5.8. Формирање контекста и позив Gemini модела	28
5.9. Кориснички интерфејс и историја	30
6. Тестирање и експериментални резултати	33
6.1. Извори доказа и поновљивост	33
6.2. Софтверски тестови	33
6.3. Провера корпуса и сегментације	34
6.4. Методологија евалуације претраге	34
6.5. Резултати основног скупа	36
6.6. Додатне језичке провере	37
6.7. Провера генерисања и времена одзива	38
6.8. Анализа грешака и ваљаности закључака	39
7. Безбедност, ограничења и даљи развој	40
7.1. Стручна одговорност и коришћење извора	40

7.2. Подаци, приступ и спољни сервиси	40
7.3. Преостала техничка ограничења	41
7.4. Приоритети за унапређење	43
8. Закључак	45
Литература	46

1. Увод

1.1. Предмет и мотивација

Рад у апотеци подразумева коришћење различитих врста стручне документације. У ту документацију спадају упутства за лекове, материјали о интеракцијама лекова и смернице добре апотекарске праксе. Иако су такви документи неопходни за проверу информација, њихово ручно прегледање може бити непрактично када корисник постави питање природним језиком и очекује да се релевантан податак пронађе у већој збирци докумената.

Проблем није ограничен на проналажење документа по називу. Једна тема може бити описана различитим изразима, док различити документи могу садржати сличне стандардизоване формулације. Назив лека, назив активне супстанце и опис питања могу упућивати на исту област, иако не представљају исти текстуални низ. Због тога је потребан поступак који повезује разумевање корисничког питања са претрагом садржаја докумената.

Велики језички модел може да формулише повезан одговор, али његово коришћење без спољног контекста не обезбеђује да је одговор заснован на конкретној документацији која је доступна у посматраном систему. *Retrieval-Augmented Generation*, односно генерисање уз помоћ претраживања, раздваја избор релевантног садржаја од његовог језичког представљања. У оригиналном раду о *RAG* приступу параметарско знање генеративног модела повезује се са непараметарском меморијом коју представља индекс спољних текстова. [1]

Предмет овог рада је развој и анализа прототипа система који наведени принцип примењује на ограничен корпус докумената релевантних за рад у апотеци. Пројекат обрађује PDF документе, препознаје њихове типове и секције, формира мање сегменте за претрагу, складишти их у *Chroma Cloud* колекцији и на основу пронађеног контекста генерише одговор помоћу *Gemini* модела. Практични опис система заснован је на стварном изворном коду и сачуваним експерименталним артефактима пројекта. [2]

1.2. Дефиниција проблема

Основни проблем који се решава јесте подршка кориснику при проналажењу релевантних делова фармацеутске документације и формулисању одговора на питање постављено природним језиком. Улаз система је питање корисника, док се одговор формира на основу садржаја који је претходно учитан и припремљен у бази знања.

Да би се овај проблем решио, потребно је обухватити више повезаних корака. Најпре се PDF документи морају учитати тако да се сачувају подаци о извору и страницама. Затим је потребно узети у обзир да различите групе докумената имају различиту структуру. Упутства за лекове садрже стандардне главне секције, Водич добре апотекарске праксе користи PDF обележиваче када су доступни, а документ о интеракцијама захтева правила за препознавање познатих наслова.

Након препознавања структуре, текст се дели на мање *child* сегменте који су погоднији за претрагу. Сваки сегмент задржава везу са широм *parent* секцијом, изворним PDF документом, страницама и, када је реч о упутству за лек, називом лека и активном супстанцом. На тај начин се резултат претраге не своди само на сличност текста, већ садржи и податке потребне за тумачење и приказ извора.

Проблем има и безбедносну и методолошку страну. Систем који проналази тематски сличан текст не мора нужно да је пронашао одговарајући лек или тачан доказ

за сваку тврдњу у одговору. Зато се резултати у овом раду тумаче као подршка претрази и читању документације, а не као аутоматска замена за стручну процену.

1.3. Циљ и обухват рад

Циљ рада је развој функционалног *RAG* прототипа који повезује обраду PDF докумената, мултијезичну векторизацију, *ChromaDB* претрагу и генерисање одговора. Посебна пажња посвећена је очувању везе између *child* сегмента и *parent* секције којој припада, јер се прецизна претрага и довољно широк контекст за генерисање не остварују нужно истим текстуалним нивоом.

Обухват реализације чине читавање PDF страница, класификација докумената, препознавање секција и формирање *parent* и *child* сегмената. *Chroma Cloud* користи густу *Qwen* и ретку *SPLADE* репрезентацију, уз комбиновање рангова методом *RRF*. *Gemini* добија питање и ограничени контекст, а *Streamlit* приказује одговор, изворе и контекст у оквиру сесије. Јединични и интеграциони тестови допуњени су оценом претраге на референтном скупу питања.

Практични допринос рада обухвата раздвајање фаза обраде, правила за различите врсте докумената, *parent-child* организацију, додавање идентификационог контекста у текст сегмената и повезивање са спољним сервисима за векторизацију, претрагу и генерисање. Додатни допринос представљају тестови и евалуација, јер омогућавају да се разликују успешан пролазак програмског тока, проналажење очекиване секције и стварна поткрепљеност генерисаног одговора.

1.4. Границе примене система

Рад не представља развој новог језичког модела нити обучавање модела од почетка. У систему се користе већ доступне компоненте: *pypdf* за читање PDF докумената, *LangChain Document* објекти за заједнички облик текста и метаподатака, *Chroma Cloud* за векторску претрагу и *Gemini* за генерисање одговора.

Прототип није клинички валидиран и није испитано да ли су сви његови одговори медицински тачни. Корпус је ограничен на затечене документе, а основни референтни скуп садржи ограничен број питања. Такође, пројекат не води евиденцију пацијената, не садржи аутентификацију и систем улога, не поставља дијагнозе и не доноси терапијске одлуке.

Систем је намењен подршци претрази и разумевању документације. Одговор треба проверити у одговарајућем изворном документу, а одлуке које захтевају фармацеутску или лекарску процену остају у надлежности квалификованог стручњака. Ова граница је важна и за тумачење резултата: мерене вредности *retrieval* метрика не могу се тумачити као клиничка тачност или безбедност система.

1.5. Организација рада

После увода следе преглед постојећих приступа и теоријске основе *RAG*-а, векторизације, хибридне претраге и генерисања. Четврто поглавље описује проблем, корпус и архитектуру, а пето конкретну имплементацију. Шесто поглавље приказује софтверске провере и резултате евалуације. Седмо разматра безбедност, ограничења и правце развоја, док осмо доноси закључак. Литература и додаци садрже коришћене изворе, инвентар корпуса, тестна питања и команде за репродукцију.

2. Преглед области и постојећих приступа

2.1. Претрага документације и генерисање одговора

За постављени проблем могу се разматрати три основна приступа. Први је непосредна претрага текстова, при којој корисник добија листу докумената или пасуса и сам саставља одговор. Други је постављање питања генеративном моделу без спољног контекста. Трећи раздваја избор доказа од његовог језичког представљања и одговара RAG архитектури. То су архитектонске алтернативе, а не три експериментално упоређена производа у овом пројекту.

Непосредна претрага има важну предност: изворни текст остаје видљив и није преформулисан моделом. Међутим, поступак проналажења, поређења и сажимања остаје на кориснику. Генерисање без претраге може дати читљив одговор, али не успоставља контролисану везу са локалном збирком PDF-ова. Избор RAG-а у овом раду мотивисан је потребом да се језичка употребљивост повеже са проналажењем проверљивих докумената.

Оригинални RAG рад описује обучиве моделе и експерименте са индексом Wikipedia пасуса. Практични систем у овом раду не репродукује тај поступак обучавања: користи већ доступне моделе и програмски организован ток претраге и генерисања. Зато је прецизније говорити о примени RAG архитектонског принципа него о имплементацији целокупног алгорита обучавања из оригиналног рада. [1]

2.2. Развој неуронског проналажења информација

Transformer архитектура увела је модел обраде секвенци заснован на механизмима пажње, без обавезног рекурентног или конволуционог језгра. Њен значај за овај рад је концептуалан: савремени модели могу формирати представе текста које зависе од околних токена. Из тога не следи да сваки модел исто представља реченицу, нити да је сваки генеративни модел непосредно добар претраживач. [3]

Sentence-BERT показује како се семантички смислене репрезентације реченица могу користити за ефикасно поређење, уместо поновног заједничког обрађивања сваког пара реченица. За систем претраге важан је принцип да се вектори докумената припреме унапред, а да се у време упита израчуна репрезентација питања. *Sentence-BERT* је овде теоријска референца; актуелни програм не користи *SentenceTransformer* за локално израчунавање вектора. [4]

Dense Passage Retrieval разматра проналажење пасуса коришћењем густих репрезентација питања и докумената у *dual-encoder* оквиру. Веза са овим пројектом није у преузимању истих тежина модела, већ у избору мањег пасуса као јединице претраге. Вектор читавог приручника тешко може једнако добро представљати све теме које се у њему појављују. Зато се у пројекту индекс претражује преко мањих сегмената. [5]

2.3. Ретке репрезентације и комбиновање рангова

Густа репрезентација није једини начин представљања текста. *SPLADE* развија ретке лексичке репрезентације са наученим тежинама и проширењем термина. Ретка репрезентација зато није исто што и једноставно бројање појављивања речи. У овом раду она служи као други сигнал претраге, уз семантички сигнал густог модела. Није спроведено локално обучавање *SPLADE* модела. [6]

Када две претраге дају различите скале оцена, њихово непосредно сабирање

захтева пажљиво усаглашавање. *Reciprocal Rank Fusion* комбинује положаје резултата на ранг-листама. Изворни рад Cormack-a, Clarke-a и Büttcher-a предлаже једноставан начин спајања више листа без обучавања новог модела рангирања. Пројекат користи пондерисану варијанту доступну у *Chroma Cloud*-у. То није исто што и *cross-encoder reranking*, јер се питање и кандидат не прослеђују додатном моделу који поново процењује њихову релевантност. [7]

2.4. Избор приступа за практични систем

Изабрана архитектура комбинује структуру извора и векторску претрагу. Познате границе секција користе се пре векторизације, док модел претраге решава који је сегмент најближи конкретном питању. Ова подела избегава захтев да *embedding* модел сам реконструише организацију читавог PDF-a.

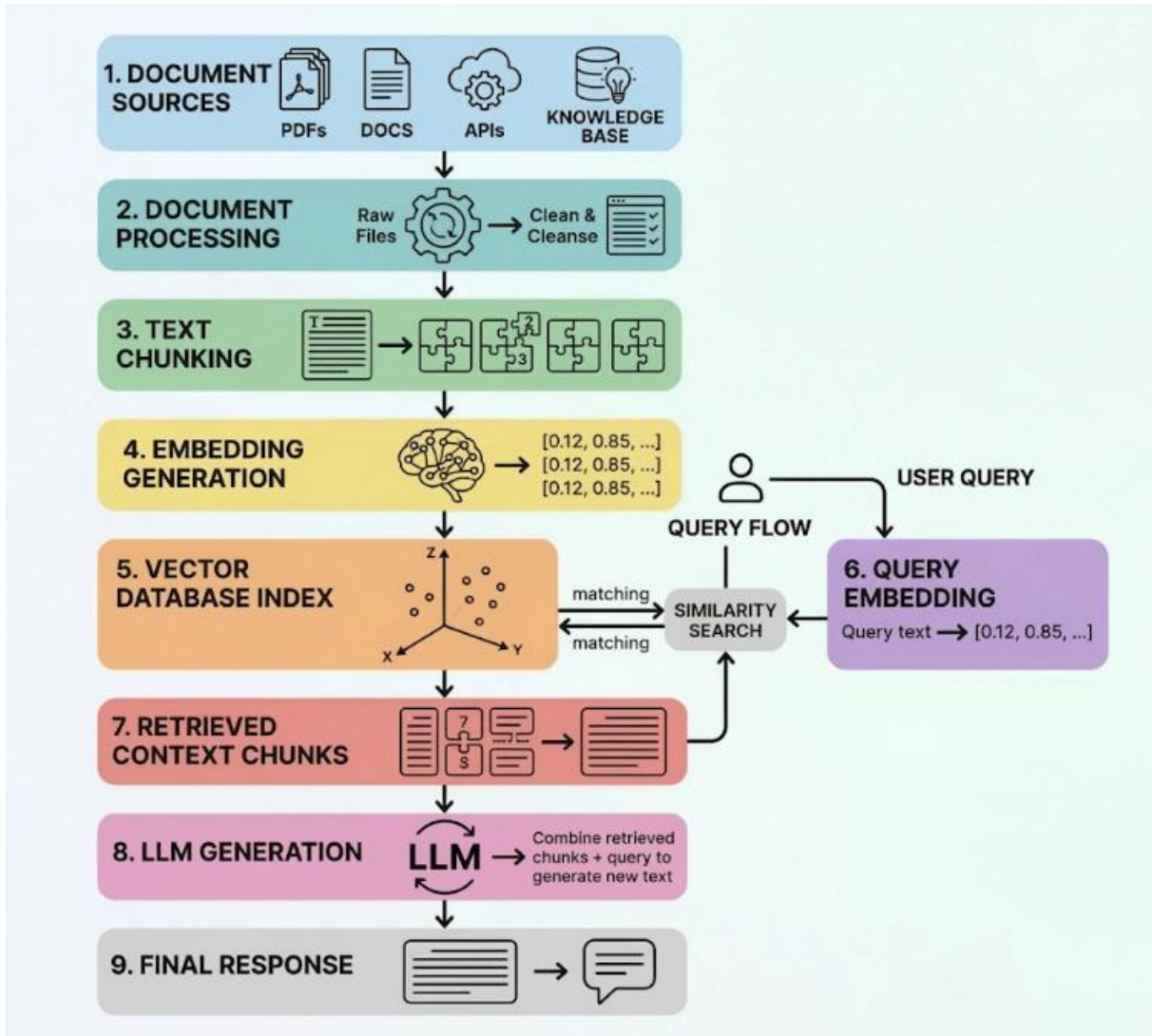
За упутство за лек посебно је важно да сегмент има идентитет лека. Текстови различитих упутстава често садрже сличне формулације и исте главне наслове. Уколико је представљено само тело пасуса, добро тематско поклапање може припадати погрешном документу. Зато у овом пројекту назив лека и активна супстанца постају део текста који се векторизује. Ова одлука је разумљива по конструкцији система, али њен изоловани допринос није измерен посебним експериментом са и без префикса.

Parent секција представља виши ниво организације документа и повезана је са својим *child* сегментима преко идентификатора *parent_id*. *Child* сегменти се користе као јединице претраге и као текстуални контекст који се прослеђује генеративном моделу. *Parent* секције се у овој варијанти не користе за замену пронађеног *child* текста целокупном секцијом, већ служе за очување структурне везе, идентификацију изворне секције и праћење порекла резултата. Оваква организација смањује количину непотребног текста у промπτу, али може довести до губитка ширег контекста који се налази ван пронађеног *child* сегмента. *Parent-child* приступ зато није аутоматско решење свих проблема сегментације, већ компромис између прецизности претраге, количине контекста и очувања шире структуре документа. Његове ефекте потребно је испитати на стварном корпусу.

3. Теоријске основе

3.1. Структура RAG система

За потребе реализације корисно је раздвојити две фазе. У фази припреме база знања се гради из докумената. У фази одговарања обрађује се појединачно питање над већ припремљеним индексом. Учитавање PDF-а и поновна векторизација целог корпуса нису потребни за свако питање.



Слика 3.1. Општа архитектура RAG система

Нека је D корпус, q питање, R поступак претраге, B припрема ограниченог контекста и G геенратор. Апликациони ток може се записати као:

$$C = B(R(q, D)), \quad a = G(q, C, I) \quad (1)$$

где је I системска инструкција, а a одговор. Формула (1) е опис декомпозиције конкретног система, а не функција за обучавање оригиналног RAG модела. Она издваја три места на којима исправан документ може бити изгубљен: претрагу, проширивање и паковање контекста.

У том смислу треба разликовати релевантан извор у корпусу, релевантан *child*

результат у претрази и релевантан доказ који је стварно достављен генеративном моделу. Чињеница да се информација налази у некој *parent* секцији не гарантује да ће бити употребљена у одговору, јер се у промпт прослеђују само изабрани *child* сегменти. Ако се тражена информација налази у другом *child* сегменту исте *parent* секције, а тај сегмент није довољно високо рангиран, он неће бити укључен у коначни контекст. Поред тога, и изабрани *child* сегменти могу бити скраћени ако укупан контекст премаши дефинисани карактерски буџет. Ова разлика је значајна за тумачење резултата практичног система.

Генеративна грешка такође није једнака *retrieval* грешци. Модел може добити исправан пасус, а да изостави услов примене, погрешно повеже две тврдње или дода непоткрепљен детаљ. Обрнуто, уверљив одговор не потврђује да је претрага успела. Због тога се у експерименталном делу одвојено разматрају исправност програмског тока, релевантност секција и квалитативно понашање генератора.

3.2. Векторизација и семантичка сличност

Векторизација текста је пресликавање текстуалног улаза у бројчану репрезентацију. За густе репрезентације може се записати $f(d) \in \mathbb{R}^m$, где је m димензија вектора. Текст и вектор нису два еквивалентна формата чувања истог садржаја: из *embedding*-а се у овом систему не реконструише оригинални пасус. Зато база задржава и текст и метаподатке.

Упит и документ морају бити представљени компатибилним поступком. Иста димензија два вектора није довољна ако потичу из различитих модела или различито конфигурисаних простора. Промена модела за документе, без одговарајуће промене репрезентације питања, нарушила би смисао поређења. То је разлог да се конфигурација *embedding* модела посматра као део верзије индекса.

У конкретној имплементацији за густ индекс користи се косинусна сличност:

$$\cos(u, v) = \frac{\sum_{j=1}^m u_j v_j}{\sqrt{\sum_{j=1}^m u_j^2} \sqrt{\sum_{j=1}^m v_j^2}} \quad (2)$$

Косинусна сличност пореди оријентацију ненултих вектора. Када API врати косинусно растојање, апликација израчунава оцену као 1.0 – *distance*. Ова оцена није вероватноћа да је одговор медицински тачан, већ служи за поређење кандидата у оквиру исте претраге. Због тога се праг прихватања резултата не може оправдати само чињеницом да нека оцена делује бројчано висока.

За репрезентацију је важан цео улазни текст, укључујући додате идентификационе линије. У конкретном пројекту питање не добија вектор читаве претходне конверзације, већ репрезентацију тренутног упита. Документ добија репрезентацију свог *page_content* поља, а не произвољних метаподатака који нису наведени као извор *embedding*-а.

3.3. Мултијезична претрага и коришћени модели

Qwen3-Embedding-0.6B је модел намењен представљању текста за *embedding* задатке. Ауторска документација наводи мултијезичну подршку за више од 100 језика, максималну димензију репрезентације 1.024 и подршку за инструкције задатка. То су својства објављеног модела, а не резултати овог дипломског експеримента. Број димензија не означава број речи или број токена који се чувају. [8]

У имплементацији бира се *QWEN3_EMBEDDING_0p6B* преко *ChromaCloudQwenEmbeddingFunction*. Ова интеграција позива удаљени *Chroma* сервис, па локални програм не учитава тежине *Qwen* модела у GPU меморију. Инструкције за документ и питање су различите, али оба улаза припадају истом *retrieval* задатку. Доступност сервиса и његова ограничења зависе од *Cloud* конфигурације. [9]

За ретке векторе користи се *ChromaCloudSpladeEmbeddingFunction* са избором *SPLADE_PP_EN_V1*. И ово израчунавање обавља удаљени сервис. Када програм позове *embedding* функцију у SDK-у, то не значи да је неуронски модел покренут у локалном *Python* процесу. [10]

Картица модела *prithivida/Splade_PP_en_v1* означава енглески језик. Зато се не може претпоставити да је његов допринос српском корпусу исти као допринос мултијезичног густог модела. [11] Практично питање је да ли ретки сигнал помаже у препознавању назива и термина, или у појединим случајевима потискује релевантан густ резултат. Одговор мора да се заснива на контролисаном поређењу.

Српска латиница, ћирилица, називи без дијакритика и енглеске парафразе нису истоветни тестни случајеви. Чак и ако модел подржава више језика, одређени назив производа може бити слабије повезан са својом транслитерацијом. У практичном коду није имплементиран регистар алијаса нити посебан корак идентификације лека. Мултијезичност модела стога представља основу за експеримент, а не гаранцију поузданости сваког упита.

3.4. Векторске базе и ChromaDB

Векторска база управља записима тако да се, поред приступа по идентификатору и метаподацима, може вршити претрага према векторској блискости. *Chroma* организује записе у колекције. Запис се у овом пројекту посматра као веза идентификатора, текста, метаподатака и векторских репрезентација. *get_or_create_collection* омогућава отварање постојеће или креирање именоване колекције. [12]

Метаподаци не служе само за приказ резултата. Филтер *record_type="child"* ограничава скуп претраге, док се *parent_id* користи за касније дохватање секције. На овај начин и *parent* и *child* записи могу постојати у истој колекцији, а да имају различите улоге. То је апликациона организација, не аутоматски релациони спој који база сама изводи.

Schema описује индексе над пољима записа. За ретки индекс конфигурише се посебан кључ, извор текста и *embedding* функција. То омогућава да један документ има густ и редак сигнал за претрагу. [13] Само присуство метаподатка *active_substance* не значи да ће претрага аутоматски бити филтрирана по тој супстанци: такав услов морао би изричито да се зада.

У пројекту се користи *Chroma Cloud*, а не локална база као подразумевано складиште. PDF-ови остају у локалном директоријуму *Literatura*, а изведени текстови и вектори чувају се у *Cloud* колекцији. Није реализована посебна релациона база пацијената, корисника или залиха.

3.5. Хибридна претрага и груписање

За *Chroma* варијанту пондерисаног RRF-а оцена може бити записана као:

$$s_{Chroma}(d) = - \sum_{i \in \{dense, sparse\}} \frac{w_i}{c + r_i(d)} \quad (3)$$

где је $r_i(d)$ ранг кандидата у одговарајућој листи, w_i тежина сигнала, а c параметар

ублажавања. Кандидат који није присутан у једној листи нема допринос из те листе, што се у формули може представити бесконачним рангом. *Chroma* документација користи рангове од нуле и негативну оцену јер мања вредност има предност. У изворном RRF раду рангови су приказани другачијом нумерационом конвенцијом; зато је при тумачењу формуле битно навести конкретан API. [14]

У пројекту су тежине густог и ретког сигнала постављене на 0,7 и 0,3, док је $c = 60$. Излазна функција мења знак *Chroma* оцене како би у апликацији већа бројчана вредност означавала бољи резултат. Ова промена знака не представља нормализацију резултата у проценат или вероватноћу. Такође, параметар $k = 60$ у RRF изразу није број коначних резултата. Број резултата који се враћа кориснику задаје се параметром `top_k`, чија је подразумевана вредност дефинисана конфигурацијом апликације.

Груписање резултата по изворном PDF-у ограничава колико погодака може потицати из истог документа. У Chroma Search API-ју GroupBy и MinK омогућавају задржавање најбољих резултата у групи. [15] У пројекту је подразумевани лимит два *child* резултата по PDF-у. То може повећати разноврсност извора, али код једног великог приручника може уклонити више корисних секција. Разноврсност докумената и потпуност доказа нису исти циљ.

3.6. Сегментација и хијерархијски контекст

Сегментација одређује који текстови постају појединачне јединице претраге. Превелик сегмент може обухватити више тема, док премали сегмент може изгубити услов примене или идентификацију предмета. Зато се у овом систему најпре користе структурне границе секција, а тек затим ограничење величине текста унутар сваке секције.

Parent документ представља логичку секцију, а *child* документ мањи део те секције. Веза између њих успоставља се метаподатком *parent_id*. У актуелној *child-only* варијанти претрага се врши над *child* текстовима, а изабрани *child* резултати се непосредно користе за формирање контекста који се прослеђује генеративном моделу. *Parent* секције се не дохватљају и не замењују пронађени *child* текст током овог процеса. Њихова улога је очување структурне везе, идентификација изворне секције и праћење порекла *child* сегмента. Сегментација и повезивање *parent* и *child* докумената реализовани су сопственим *Python* кодом; није употребљена *LangChain* класа *ParentDocumentRetriever*.

Преклапање суседних сегмената преноси део текста са краја претходног сегмента на почетак наредног. Тиме се умањује губитак локалног контекста, али се повећава број поновљених речи и потенцијално сличних погодака. Само преклапање не гарантује очување целе реченице или структуре табеле. То мора бити својство сваке гране алгоритма за поделу текста.

Поред семантичке поделе, постоји и техничко ограничење складишта. *Chroma* наводи ограничење од 16 KiB за појединачни текстуални запис. То се односи на број бајтова, а не на број токена или на 16.384 српска знака. [16] Пројекат зато има додатну поделу према величини UTF-8 записа. *Parent* секција може логички остати једна целина, иако се физички чува у више *Cloud* записа који се могу поново спојити када се *parent* запис експлицитно дохвати. У актуелној *child-only* путањи ово поновно спајање *parent* делова није потребно за формирање контекста који се шаље генеративном моделу.

3.7. Велики језички модел и генерисање

Велики језички модел генерише текст на основу улазног низа токена и параметара стечених обучавањем. Токен може бити реч, део речи или други фрагмент записа. Зато број знакова које апликација шаље није исто што и број токена које модел обрађује. За овај пројекат значајно је да се различито мере тело сегмента, UTF-8 величина документа, величина контекста и максимална дужина одговора.

У прегледаном коду подразумевани генеративни модел је *gemini-3.6-flash*, што одговара и ознаци у сачуваном експерименту. *Google* објављује овај идентификатор у документацији модела. [17] У раду се не претпоставља архитектура скривених слојева, број параметара или интерна ревизија *Cloud* модела које апликација не бележи.

Програм користи *Google Gen AI SDK* и *Interactions API*, са улазним промптом, системском инструкцијом и ограничењем излазних токена. Промпт садржи корисничко питање и одабране *child* сегменте добијене *retrieval* поступком. *Parent* секције се не прослеђују моделу као замена за пронађене *child* текстове. API подржава различите начине интеракције, али сам избор овог API-ја не значи да апликација користи агенте или меморију разговора. Конкретан позив не шаље претходну интеракцију и има подешавање *store=False*.

Системска инструкција захтева одговор на српском језику, употребу достављеног контекста и признавање недостатка информација. Ово усмерава модел, али не представља формалну проверу сваке генерисане тврдње. Генератор у овом пројекту нема додатни валидатор који упоређује дозе, називе и услове са изворним документом. Та разлика између инструкције и провереног ограничења пресудна је за оцену безбедности система.

4. Анализа проблема и пројектовање система

4.1. Корисници, задаци и границе примене

Предвиђени корисник прототипа је особа која тражи податке у фармацеутској документацији и која може да провери добијени одговор. У контексту теме рада то је пре свега фармацеут, док демонстрацију могу користити ментор, чланови комисије и други испитивачи. У програму не постоје посебне корисничке улоге; ово је опис намене система, а не имплементирани систем ауторизације.

Типичан задатак је проналажење секције о примени, упозорењима, чувању или нежељеним дејствима наведеног лека. Другу групу чине питања о појмовима и механизмима интеракција, а трећу питања о поступцима описаним у водичу добре апотекарске праксе. Код прве групе доминирају идентификација лека и тема; код друге појам или механизам; код треће одговарајући поступак и његова позиција у хијерархији приручника.

Упутство за пацијента и сажетак карактеристика лека не треба сматрати истим типом документа. ЕМА QRD образац разликује ове документе и њихове структуре. Шест секција које програм препознаје одговара структури упутства за пацијента, а не представља општи парсер свих врста медицинске документације. [19] Ова референца објашњава структурни образац, али не доказује регулаторни статус сваког локалног PDF документа.

У актуелној *child-only* варијанти систем претражује *child* сегменте и те сегменте користи за формирање контекста који се прослеђује генеративном моделу. *Parent* секције служе за организацију корпуса и очување везе са изворном секцијом, али се њихов целокупан текст не прослеђује моделу као замена за пронађени *child* сегмент.

Граница примене постављена је на претрагу докумената и пружање информација из доступног корпуса. Систем не издаје рецепте, не врши проверу терапије над потпуним картоном пацијента, не управља залихама и не приступа апотекарском информационом систему. Питања која захтевају анамнезу, лабораторијске податке или процену хитности не могу се поуздано решити само на основу пронађених општих упутстава и *child* сегмената.

4.2. Структура и порекло корпуса

Примарни улаз у систем представља локални директоријум *Literatura*. Након читавања PDF докумената, документи се организују у *parent* секције и *child* сегменте, који представљају основне јединице за даљу индексацију и претрагу.

Табела 1. Састав корпуса и резултат структурне припреме

Тип	Број PDF-ова	Стр.	parent секције	child сегменти
Упутства за лекове	29	226	203	741
Водич добре апотекарске праксе	1	93	279	536
Референца о интеракцијама	1	20	11	84
Укупно	31	339	493	1.361

Упутства за лекове коришћена у корпусу преузета су са званичне странице

Агенције за лекове и медицинска средства Србије (АЛИМС). Водич добре апотекарске праксе преузет је са веб-сајта Министарства здравља Републике Србије, док је референтни документ о интеракцијама лекова преузет са веб-сајта Фармацеутске коморе Србије. [20.1–20.3]

4.3. Функционални и нефункционални захтеви

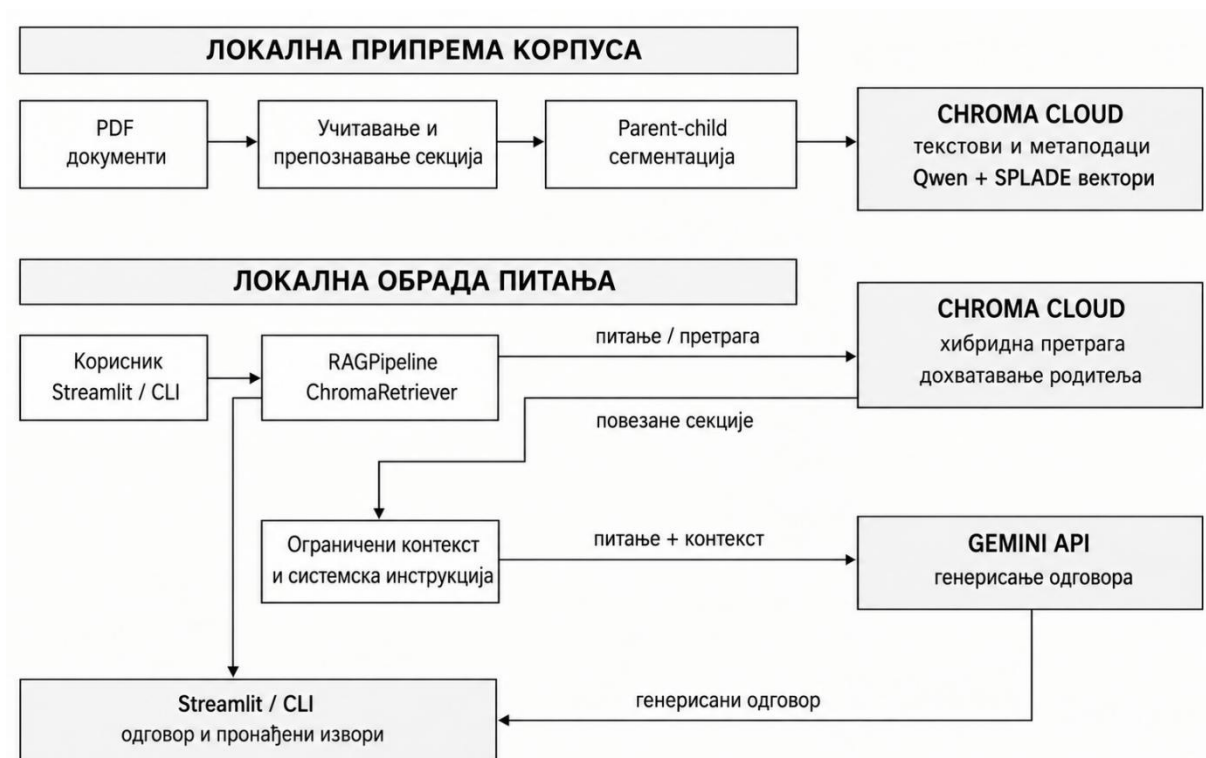
Табела 2. Функционални захтеви и њихова реализација

Захтев	Реализовани механизам
Учитавање PDF докумената из стабла директоријума	Учитавање PDF докумената из стабла директоријума
<code>load_all_documents</code>	<code>load_all_documents</code>
Разликовање типова докумената	Разликовање типова докумената
<code>DocumentType</code> и <code>detect_document_type</code>	<code>DocumentType</code> и <code>detect_document_type</code>
Подела докумената према структури извора	Подела докумената према структури извора
<code>build_parent_sections</code>	<code>build_parent_sections</code>
Формирање <code>parent</code> секција и <code>child</code> сегмената	Формирање <code>parent</code> секција и <code>child</code> сегмената
<code>build_parent_child_documents</code> , <code>parent_id</code> и метаподаци сегмената	<code>build_parent_child_documents</code> , <code>parent_id</code> и метаподаци сегмената
Претрага <code>child</code> сегмената уз коришћење метаподатака извора	Претрага <code>child</code> сегмената уз коришћење метаподатака извора
<code>ChromaVectorStore.search</code> , <code>dense/sparse</code> индекси и <code>CHILD_FILTER</code>	<code>ChromaVectorStore.search</code> , <code>dense/sparse</code> индекси и <code>CHILD_FILTER</code>

Нефункционални захтеви односе се на разумљивост кода, следљивост резултата, поновљивост припреме и контролисану употребу спољних сервиса. Модуларна организација и типизовани објекти доприносе одржавању система. Идентификатори извора, секције и *parent-child* везе подржавају праћење порекла пронађених *child* сегмената. Конфигурациони параметри издвојени су у посебан модул, што олакшава промену вредности које се често подешавају. Тестови омогућавају проверу уговора између модула без неопходног позива удаљених *embedding* или генеративних модела.

4.4. Архитектура и границе одговорности

Систем је модуларна *Python* апликација са *Streamlit* интерфејсом и два спољна сервиса. Прегледач комуницира са *Streamlit* сервером, а серверски *Python* код непосредно позива *Chroma* и *Google SDK*. CLI користи исте модуле без прегледача.



Слика 4.1. Архитектура реализованог система, ауторска текстуална шема на основу кода

Раздвајање модула омогућава проверу сваке фазе. Парсер секција може да се испита над локалним страницама без базе. Функције за припрему *Chroma* записа могу да се тестирају над синтетичким *Document* објектима. *Retriever* и *RAGPipeline* могу да добију лажне зависности. То је конкретна предност архитектуре за дипломски пројекат: узрок грешке може се тражити на одређеној граници уместо у једној великој функцији.

4.5. Модел података и следљивост

LangChain се користи кроз стандардну репрезентацију текста и метаподатака у *Document* објекту. То омогућава да странице, секције и сегменти деле исти основни интерфејс. [21] У овој апликацији *loader* је сопствена функција, а не тврдња да је наслеђена одређена *LangChain loader* класа.

Значење објекта зависи од фазе. *Document* који излази из *loader*-а представља једну непразну страницу. *Parent Document* представља препознату секцију. *Child Document* представља мањи део секције са префиксом. Стога назив *page_content* не значи да сваки каснији објекат мора одговарати физичкој PDF страници.

Табела 3. Најважнији метаподаци и њихово значење

Поље	Ниво и улога
<i>source, file_name, folder</i>	Путања, назив PDF извора и назив родитељског директоријума
<i>source_document_id</i>	Идентификација изворног PDF документа; заједничка је за записе који потичу из истог документа

Поље	Ниво и улога
<i>document_type</i>	Одређује породицу правила која се користи за препознавање структуре документа
<i>medicine_name, active_substance</i>	Подаци из имена датотеке и директоријума; користе се првенствено за упутства за лекове
<i>page, page_number, page_index</i>	Број и индекс PDF странице; у каснијим фазама могу бити наслеђени од изворних <i>page-level</i> записа
<i>page_numbers</i>	Странице које обухвата једна <i>parent</i> секција
<i>section_id, parent_id</i>	Идентификатор секције и веза <i>child</i> записа са одговарајућом <i>parent</i> секцијом
<i>section_title, section_path</i>	Наслов секције и њена хијерархијска путања у документу
<i>section_detection</i>	Начин на који је препозната граница секције
<i>record_type</i>	<i>Тип записа: parent или child</i>
<i>child_chunk_index, child_chunk_count</i>	Редни број <i>child</i> сегмента и укупан број <i>child</i> сегмената у <i>parent</i> секцији
<i>cloud_subchunk_index</i>	Редни број физичког дела текста када је запис због ограничења величине подељен на више <i>Cloud</i> записа

4.6. Пројектне одлуке и алтернативе

Прва одлука је издвајање логичке структуре документа пре примене ограничења дужине текста. Када је тип документа познат, границе главних секција користе се за формирање *parent* секција. Тек се унутар тих секција примењује ограничење величине којим се формирају *child* сегменти погодни за претрагу. Наслови водича не третирају се истим правилима као шест секција упутства за лек. Ова одлука повећава зависност *parser*-а од типа корпуса, али његово понашање чини објашњивијим. Алтернатива би била примена једног општег делиоца текста на све документе, али би се тиме изгубила део информација о изворној структури.

Друга одлука је употреба идентификационих префикса у самом садржају сегмента. Ако би назив лека био сачуван само као метаподатак који није укључен у текстуални извор *embedding*-а, модел га не би нужно узимао у обзир приликом векторизације. Префикс зато у текст уноси идентитет лека, активну супстанцу, назив документа и секцију када су ти подаци доступни. Цена ове одлуке су додатни знакови у сваком запису и могућност да дугачак префикс несразмерно утиче на веома кратак сегмент.

Трећа одлука је коришћење *Cloud* векторизације. *Dense* и *sparse embedding* функције извршавају се у *Chroma Cloud*-у, чиме се избегава локално учитавање модела и њихових тежина у интерактивном раду. Са друге стране, ова одлука уводи зависност од мреже, спољну обраду текста и трошак коришћења сервиса. У пројекту нису реализовани локални *fallback embedding* модел нити аутоматска замена *Cloud* претраге локалним индексом.

Четврта одлука је чување *parent* и *child* записа у истој *Chroma* колекцији. То поједностављује конфигурацију колекције, управљање записима и очување *parent-child* везе. Међутим, пошто се у истој колекцији чувају обе врсте записа, *embedding* обрада се

примењује и на *parent* и на *child* текстове. Стандардна претрага се филтрира на *child* записе помоћу метаподатка *record_type="child"*, док се у актуелној *child-only* варијанти пронађени *child* текст непосредно користи за формирање контекста који се прослеђује генеративном моделу. *Parent* записи остају сачувани ради организације, праћења порекла и очувања везе преко *parent_id*, али се њихов целокупан текст не користи као замена за пронађени *child* сегмент. Алтернатива би била чување *parent* секција у засебном текстуалном складишту или засебној колекцији.

Пета одлука је избор једноставног секвенцијалног тока уместо агентског приступа. Број основних корака је познат: претрага *child* сегмената, припрема ограниченог контекста и један генеративни позив. То олакшава тестирање, праћење података и анализу грешака. Истовремено, систем нема аутоматско постављање допунског питања, нову претрагу на основу непотпуног или празног резултата, нити проверу генерисаних тврдњи другим моделом.

5. Имплементација система

5.1. Развојно окружење и организација кода

Пројекат је реализован у програмском језику *Python*, уз коришћење библиотека *chromadb*, *google-genai*, *httpx*, *langchain-core*, *python-dotenv*, *pypdf* и *streamlit*. Коришћене верзије кључних библиотека су: *chromadb 1.5.9*, *google-genai 2.21.0*, *langchain-core 1.5.3*, *pypdf 6.15.0* и *streamlit 1.63.0*.

Изворни код је организован по функционалним целинама. Модули за учитавање и анализу докумената одвојени су од модула за сегментацију, припрему *Chroma* записа, претрагу и генеративно одговарање. Оваква организација омогућава да се појединачне фазе система тестирају и анализирају независно.

Табела 4. Преглед модула и њихових одговорности

Модул	Одговорност
<code>data_loader.py</code>	Учитавање PDF страница, издвајање текста и формирање основних метаподатака
<code>document_types.py</code>	Препознавање типа документа и избор одговарајуће породице правила за даљу обраду
<code>document_sections.py</code>	Препознавање и формирање логичких <i>parent</i> секција на основу структуре документа
<code>chunking.py</code>	Формирање <i>child</i> сегмената унутар <i>parent</i> секција, додавање <i>retrieval</i> префикса и успостављање <i>parent-child</i> везе.
<code>chroma_documents.py</code>	Припрема записа за <i>Chroma Cloud</i> , контрола величине текста, чишћење метаподатака и формирање идентификатора
<code>vector_store.py</code>	Креирање <i>Chroma Cloud</i> колекције, <i>dense</i> и <i>sparse</i> индекса, упис записа и претрага <i>child</i> сегмената
<code>retriever.py</code>	Танак <i>retrieval</i> слој који прослеђује питање складишту и резултате претвара у објекте <i>RetrievedDocument</i>
<code>rag_prompt.py</code>	Формирање контекста од <i>retrieved child</i> сегмената, припрема <i>prompt</i> -а и издвајање информација о изворима
<code>gemini_client.py</code>	Позив <i>Google Gen AI SDK</i> -а и <i>Gemini</i> модела за генерисање одговора
<code>rag_pipeline.py</code>	Оркестрација целокупног RAG тока: <i>retrieval</i> , формирање <i>prompt</i> -а и генерација одговора
<code>ui_app.py</code>	Реализација <i>Streamlit</i> интерфејса, унос питања, приказ одговора, извора и <i>retrieved</i> контекста

5.2. Учитавање PDF докумената

Функција *load_pdf* најпре проверава да ли прослеђена путања постоји, да ли представља датотеку и да ли има PDF екстензију. Затим отвара документ помоћу класе *PdfReader*, издваја узорак текста са прве две странице ради препознавања типа документа и пролази кроз све странице PDF-а. За сваку страницу са успешно издвојеним и непразним текстом формира се један почетни, *page-level* објекат *Document*.

Поступак издвајања текста једноставан је и видљив у листингу 5.1. Функција користи постојећи текстуални слој PDF документа и не покреће OCR. Документација библиотеке *pypdf* разликује издвајање текста из PDF-а од препознавања текста који се налази у слици. [22]

Листинг 5.1. Издвајање текста странице из *data_loader.py*

```
def _extract_page_text(page: Any) -> str:
    try:
        return (page.extract_text() or "").strip()
    except Exception as exc:
        print(f"[WARN] Could not extract page text: {exc}")
        return ""
```

Ако издвајање текста не успе, функција враћа празан стринг. На тај начин тренутна страница се прескаче, а обрада осталих страница може да се настави. Слично томе, функција *load_all_documents* хвата грешку на нивоу појединачног PDF документа, исписује упозорење и наставља обраду следећег документа. Овакво понашање је практично за демонстрациони *ingestion*, али може довести до непотпуног корпуса без статуса који би означио да је цела команда неуспешна. Због тога је потребно пратити упозорења и упоредити број очекиваних са бројем успешно обрађених датотека и страница.

Функција *detect_document_type* одређује тип документа на основу његове путање и узорка текста. Редослед правила је значајан: најпре се проверавају обележја водича и документа о интеракцијама, затим присуство директоријума *Lekovi*, а потом текстуални маркер који указује на упутство за лек. Документ који не одговара ниједном од ових правила добија тип *unknown*. Ова класификација је хеуристичка, јер се заснива на називу путање и узорку текста, а не на обученом класификатору.

Назив лека добија се из назива PDF датотеке без екстензије. Подвлаке и цртице претварају се у размаке, а прво слово се, када је потребно, пише великим словом. Активна супстанца добија се из назива непосредног родитељског директоријума. На тај начин организација локалних датотека добија употребљиву улогу у формирању метаподатака и *retrieval* префикса. Ипак, у систему нису реализовани аутоматска корекција правописа, препознавање синонима нити фармацеутска валидација назива.

У текућем корпусу свака страница дала је непразан текст. Тај резултат показује да је постојао читљив текстуални слој.

5.3. Препознавање секција

Функција *build_parent_sections* групише странице према изворном PDF документу и уређује их по редоследу страница. Сваки PDF обрађује се независно, па *parent* секција не може настати спајањем краја једног и почетка другог документа. Текстови страница се затим повезују, док *TextSpan* објекти памте позиције у комбинованом тексту и бројеве страница на које се ти делови односе.

За упутства за лекове користе се шаблони шест главних наслова. Нормализовани текст олакшава поређење различитих размака, интерпункције и латиничких дијакритичких знакова, а посебни шаблони обухватају и ћириличне формулације. Поред нормализованог текста, чува се мапа оригиналних позиција. То је важно зато што се коначна *parent* секција издваја из оригиналног текста, а не из нормализоване копије.

Упутства често у уводном делу набрајају наслове који се касније понављају као стварне границе поглавља. Функција *_select_last_complete_leaflet_sequence* зато уназад тражи последњи потпуни редослед од шесте до прве секције. Она не бира прво појављивање сваког броја. Текст испред прве изабране секције остаје као *Document introduction*. У провереном корпусу свако од 29 упутстава на тај начин даје седам *parent* секција: увод и шест главних секција.

Овај алгоритам је детерминистички, али има услове успешности. Потребно је да свих шест образаца буде пронађено и да њихов редослед одговара претпостављеној структури. Ако један од наслова недостаје, користи се подела по страницама. Чињеница да је препознато тачно шест наслова није довољан доказ да су све границе и целокупни садржај секција исправно издвојени. Она представља структурну проверу коју треба допунити садржинском анализом.

За водич се чита PDF *outline*, односно скуп обележивача. Рекурзивним проласком кроз *outline* бележе се наслов, страница и путања предака. Затим се на наведеној страници тражи одговарајући текстуални наслов. Ако је наслов дугачак, код испробава и краће кандидате. Уколико нема употребљивих обележивача или није могуће повезати обележивач са текстуалним насловом, примењује се *fallback* подела по страницама.

За документ о интеракцијама користи се листа познатих назива, као што су *Vrste interakcija*, *Farmakokineticke interakcije*, *Resorpcija*, *Metabolizam* и *Interakcije lekova sa hranom*. Провера је усмерена на самосталне линије које имају облик наслова. Ова правила су прилагођена конкретном документу и не треба их описивати као универзално препознавање секција у свим фармаколошким приручницима.

Препознате секције представљају *parent* ниво структуре документа. У наредној фази се свака *parent* секција, ако је потребно, дели на мање *child* сегменте применом ограничења дужине и преклапања.

5.4. Parent-child сегментација

Функција *build_parent_child_documents* најпре обрађује већ формиране *parent* секције. За сваку секцију припрема се текст са *retrieval* префиксом, док се оригинално тело секције прослеђује функцији *chunk_parent_sections*. На тај начин се логички садржај секције дели на *child* сегменте пре додавања префикса сваком појединачном сегменту.

Подразумевана величина тела *child* сегмента износи 1.000 знакова, док је максимално преклапање постављено на 200 знакова. Функција *split_text* покушава да текст подели на границама пасуса, редова и речи. При спајању издвојених јединица настоји се да се очува читљив текст, док се део завршетка претходног сегмента може пренети на почетак наредног. Ефективна величина преклапања зависи од размака између јединица текста и конкретне гране извршавања. Због тога није прецизно тврдити да се свака два узастопна *child* сегмента преклапају са тачно 200 знакова.

У систему је реализована структурно вођена подела уз коришћење сопственог текстуалног *splittera*. *Parent* секције постављају границе између логичких целина, док се *child* сегменти формирају унутар тих граница.

Сваки *child* сегмент наслеђује идентитет изворног PDF документа и *parent* секције. Додатно му се придружују редни број сегмента унутар секције и укупан број сегмената те секције. На тај начин сваки резултат претраге може да се повеже са изворним документом, *parent* секцијом и својом позицијом унутар те секције.

Retrieval префикс је део поља *page_content*, па улази и у *dense* и *sparse* репрезентацију које се формирају над текстом документа. Он остаје доступан и приликом приказа пронађеног сегмента. На тај начин назив лека, активна супстанца и други идентификациони подаци постају део текста који се користи у векторској претрази.

Последица додавања префикса је да укупан текст *child* записа може бити дужи од 1.000 знакова, иако је ограничење примењено на тело секције пре додавања префикса. У измереном корпусу најдужи *child* запис са префиксом имао је 1.366 знакова. Наведена вредност односи се на укупан текстуални садржај записа, а не само на његово подељено тело.

5.5. Припрема записа за Chroma Cloud

Функција *prepare_documents_for_chroma* прима већ формиране *Document* објекте. Њена одговорност није препознавање типа документа, избор секције или формирање *child* сегмената, већ техничка припрема записа за упис у *Chroma Cloud*. За сваки улазни текст функција проверава да ли његова UTF-8 величина премашује дозвољено ограничење. Ако је потребно, текст дели на више физичких *Cloud* записа, допуњује њихове метаподатке и враћа припремљену листу докумената.

Ограничење је постављено на 16*1024, односно 16.384 бајта по текстуалном запису. За ASCII текст број знакова и број бајтова се поклапају, али ћирилички знакови и латиничка слова са дијакритицима могу заузети више бајтова у UTF-8 кодирању. Због тога провера *len(text)*, није замена за проверу *len(text.encode("utf-8"))*.

Код најпре покушава да групише целе линије. Ако једна линија сама премашује дозвољену величину, дели је по *Unicode* знаковима, при чему се величина сваког дела одређује на основу његове UTF-8 репрезентације. На тај начин се физички записи задржавају унутар ограничења броја бајтова, а не само унутар ограничења броја знакова.

Сваки физички део добија вредност *cloud_subchunk_index*, која означава његов редни број унутар првобитног записа. Ово поље не представља нову логичку *child* сегментацију, већ само техничку поделу текста ради ограничења величине *Cloud* записа.

Када се предугачак *parent* запис подели на више физичких делова, ти делови задржавају исти *parent_id*. На основу тог идентификатора функција *format_parent_records* може да их сортира по пољу *cloud_subchunk_index* и поново повеже у целину.

5.6. Векторизација, колекција и упис

Chroma клијент добија *host*, *tenant*, *database* и API кључ из конфигурације окружења. Вредности API кључева нису део рукописа. Подразумевани назив колекције у коду је *diplomski_rag*. За ову колекцију *schema* дефинише *dense* индекс над текстом документа и *sparse* индекс под кључем *sparse_embedding*.

Листинг 5.2 приказује конфигурацију *dense_embedding* функције. Инструкције су дефинисане у речнику *QWEN_RETRIEVAL_INSTRUCTIONS*: једна инструкција описује текст документа, а друга корисничко питање за проналажење релевантних пасуса о лековима, интеракцијама и апотекарској пракси.

Листинг 5.2. Избор *Cloud embedding* модела из *vector_store.py*

```
def qwen_embedding_function() -> ChromaCloudQwenEmbeddingFunction:
    """Dense embedding function executed by Chroma Cloud."""

    return ChromaCloudQwenEmbeddingFunction(
        model=ChromaCloudQwenEmbeddingModel.QWEN3_EMBEDDING_0p6B,
        task=DEFAULT_CHROMA_QWEN_TASK,
        instructions=QWEN_RETRIEVAL_INSTRUCTIONS,
    )
```

Приликом позива *get_or_create_collection()* прослеђује се *schema* објекат који описује структуру колекције и њене индексе. У конкретном пројекту *schema* садржи *dense* индекс са *Qwen embedding* функцијом и *sparse* индекс са *SPLADE embedding* функцијом. Оба индекса користе текст из поља *DOCUMENT*, док је *sparse* индекс доступан под кључем *sparse_embedding*. Истовремено се прослеђује *embedding_function=None*. Разлог је то што су *embedding* функције већ дефинисане у конфигурацији одговарајућих индекса унутар *schema* објекта. На тај начин се избегава истовремено задавање *embedding* функције у *schema*-и и на нивоу колекције, што би могло довести до конфликта у конкретној верзији *Chroma SDK*-а.

Упис се врши серијски, у пакетима од 16 записа. Апликација *Chroma Cloud*-у доставља идентификаторе, текстове и очишћене метаподатке, док конфигуриране *Cloud embedding* функције на основу текстуалног поља *DOCUMENT* формирају *dense* и *sparse* репрезентације.

Листинг 5.3. Упис једног пакета из *ChromaVectorStore.upload_documents*

```
self.collection.upsert(
    ids=[document_id(document, start + index) for index, document in enumerate(batch)],
    documents=[document.page_content for document in batch],
    metadatas=[clean_metadata(document.metadata) for document in batch],
)
```

Функција *upsert* омогућава унос новог записа или измену постојећег записа са истим идентификатором. Ако се због промењеног редоследа обраде или измењених улазних података израчуна нови идентификатор, стари запис се тиме не уклања аутоматски. Због тога потпуно освежавање колекције и инкрементални упис нису исти поступак.

Команда са опцијом *--reset* брише изабрану *Cloud* колекцију и поново је креира, након чега у њу уписује припремљене записе. Овај поступак обезбеђује чисту изградњу колекције, али није атомска замена постојеће верзије. Ако се упис прекине, колекција може остати делимично попуњена.

5.7. Претрага и формирање *retrieval* излаза

Претрага у пројекту реализована је у класи *ChromaVectorStore*, у датотеци *vector_store.py*. Основна метода је *search()*, која прима текст корисничког упита и враћа најрелевантније *child* записе из *Chroma Cloud* колекције.

Претрага се врши над *child* записима, јер они представљају мање текстуалне јединице погодније за прецизно проналажење релевантног садржаја. Пре покретања претраге проверава се да ли колекција садржи *child* записе. Уколико *child* записи не постоје, метода враћа празну листу и претрага се не извршава.

Након тога одређује се број кандидата који ће бити разматрани у почетној фази претраге:

$$\text{limit} = \min(\max(\text{top_k}, \text{candidate_count}), \text{child_count}) \quad (2)$$

Параметар *top_k* одређује највећи број *child* резултата у коначној листи, док *candidate_count* одређује ширину почетног скупа кандидата. Подразумевана вредност параметра *candidate_count* је 60. То значи да се у почетној фази може разматрати до 60 *child* записа, у зависности од њиховог укупног броја. Ова вредност не представља број резултата који ће бити приказани кориснику, нити број сегмената који се обавезно прослеђује генеративном моделу.

Када је укључена хибридна претрага, пројекат истовремено користи *dense* и *sparse* претрагу. *Dense* део користи подразумевани *dense* индекс, док се *sparse* део позива преко кључа *sparse_key*. За оба начина претраге користи се исти текст упита. Хибридни ранг конструише се помоћу два *Knn* израза и *Rrf* механизма, као што је приказано у листингу 5.4.

Листинг 5.4. Конструисање хибридног ранга из *vector_store.py*

```
rank = Rrf(
    ranks=[
        Knn(query=query_text, limit=limit, return_rank=True),
        Knn(query=query_text, key=self.sparse_key, limit=limit, return_rank=True),
    ],
    weights=[self.dense_weight, self.sparse_weight],
    k=60,
)
```

Први *Knn* израз односи се на *dense* претрагу, док други користи *sparse* индекс регистрован под кључем *sparse_embedding*. *Dense* и *sparse* претрага формирају засебне ранг-листе, а *Rrf* их комбинује на основу позиција резултата у тим листама. Подразумеване тежине су *dense* = 0.7 и *sparse* = 0.3. На тај начин *dense* резултати имају већи утицај на комбиновани ранг, док *sparse* претрага додатно доприноси проналажењу записа који садрже важне термине из упита.

Хибридни ранг се затим повезује са филтером за *child* записе. Иако су *parent* и *child* записи смештени у истој колекцији, примарна претрага се врши само над записима чија је вредност метаподатка *record_type* једнака *child*. На тај начин *parent* записи не учествују као самостални кандидати у активној *retrieval* путањи.

Уколико је укључено груписање по изворном документу, претрага се додатно конфигурише на следећи начин:

Листинг 5.5. Груписање по изворном документу

```
search = search.group_by(
    GroupBy(
        keys=K("source_document_id"),
        aggregate=MinK(
            keys=K.SCORE,
            k=self.group_by_document_limit,
        ),
    )
)
```

Груписање се врши на основу поља *source_document_id*, односно идентификатора изворног PDF документа. За сваки изворни документ задржава се ограничен број најбоље ранжираних *child* резултата. Подразумевана вредност је два *child* резултата по изворном документу. Овај механизам повећава разноврсност извора, али може ограничити број корисних сегмената који потичу из једног великог документа.

Хибридна претрага извршава се унутар *try* блока. Ако током њеног извршавања дође до изузетка, програм исписује упозорење и покушава *dense* претрагу као резервну путању.

Листинг 5.6. Резервна *dense* путања за хибридную претрагу

```
try:
    results = self.hybrid_search(query_text, limit, should_group)
except Exception as exc:
    print(f"[WARN] Hybrid search failed; falling back to dense query: {exc}")
    results = self.dense_search(query_text, limit)
```

Изабрани *child* резултати остају у свом изворном облику и њихов текст се користи за формирање контекста који се прослеђује генеративном моделу. Метаподатак *parent_id* и даље остаје доступан како би се *child* резултат повезао са *parent* секцијом, изворним документом и логичком структуром корпуса.

Оцена приказана уз резултат представља оцену *retrieval*-а за пронађени *child* запис. Она није ново израчуната сличност целе *parent* секције. Метаподаци као што су *retrieved_child_id*, *retrieved_child_score* и *retrieved_child_chunk_index* могу да сачувају информације о пронађеном *child* сегменту и његовој позицији. На крају се враћа највише *top_k* *child* резултата. Ако је укључено груписање или ако је број доступних *child* записа мањи од задатог лимита, коначна листа може садржати мање резултата од вредности *top_k*.

ChromaRetriever не учитава PDF документе, не извршава поновну сегментацију и не реконструише *parent* секције. Он прослеђује питање слоју *ChromaVectorStore* и резултате претвара у листу објеката типа *RetrievedDocument*. На тај начин *RAGPipeline* не мора да познаје интерну структуру *Chroma* одговора.

Листинг 5.7. Класа *RetrievedDocument*

```
@dataclass
class RetrievedDocument:
    text: str
    metadata: dict[str, Any]
    score: float
    document_id: str | None = None
    chroma_score: float | None = None
    distance: float | None = None
    search_type: str | None = None
```

5.8. Формирање контекста и позив *Gemini* модела

Метода *RAGPipeline.answer()* представља завршну фазу RAG тока. Њена улога је да прими питање корисника, преузме релевантне *child* сегменте преко *retriever* компоненте, формира *prompt* и, уколико је пронађен непразан контекст, проследи захтев *Gemini* моделу. Резултат обраде је објекат типа *RAGResponse*, који обједињује питање, генерисани одговор, изворе, пронађене документе и коришћени *prompt*.

У првом кораку метода *answer()* прослеђује питање *retriever* компоненти. *Retriever* враћа листу објеката типа *RetrievedDocument*. Поље *text* у тим објектима представља текст пронађеног *child* сегмента.

Након завршетка *retrieval* фазе проверава се да ли је враћена листа празна. Ако *retrieval* не врати ниједан резултат, *Gemini* модел се не позива. Систем тада директно враћа унапред дефинисану поруку којом обавештава корисника да у доступним документима није пронађен контекст потребан за генерисање одговора.

Ако *retrieval* врати непразну листу резултата који нису довољно релевантни, не постоји посебна детерминистичка провера или праг који би такве резултате аутоматски

одбацио. Они се прослеђују *Gemini* моделу, а од модела се очекује да на основу системске инструкције процени да ли је достављени *child* контекст довољан за одговор. Ако контекст не подржава одговор, модел треба да наведе да у доступним документима нема довољно информација.

Ако листа пронађених докумената није празна, формира се коначни *RAG prompt*.

Листинг 5.8. Формирање *RAG prompt*-а

```
prompt = build_rag_prompt(  
    question=question,  
    documents=retrieved_documents,  
    max_context_chars=self.max_context_chars,  
)
```

Функција *build_rag_prompt()* најпре позива функцију *format_context()*, која *retrieved* резултате претвара у текстуални блок контекста. За сваки пронађени *child* сегмент формира се посебан блок. Он садржи редни број резултата, *retrieval* оцену, ознаку извора, информације о леку и активној супстанци када постоје, назив секције, тип записа и текст *child* сегмента.

Након блока са контекстом додају се питање корисника и инструкција за генерисање одговора. На тај начин се у једном *prompt*-у обједињују пронађени *child* контекст, питање корисника и инструкције за генерисање одговора.

Количина текста која се укључује у *prompt* ограничена је параметром *max_context_chars*. Његова подразумевана вредност у пројекту износи 14.000 знакова. Ова вредност представља апликационо ограничење текста *retrieval* контекста, а не максималну величину контекстног прозора *Gemini* модела. Ако сви *retrieved child* сегменти не могу да стану у расположиви буџет, контекст се ограничава према редоследу резултата и расположивом броју знакова.

Поред главног *prompt*-а, *Gemini* модел добија и системску инструкцију дефинисану у датотеци *rag_prompt.py*. Системска инструкција одређује понашање модела и захтева да модел одговара на српском језику, користи достављени контекст и не измишља медицинске информације. Такође, модел треба да наведе када достављени контекст није довољан и да генерисани одговор не представља замену за савет лекара или фармацеута.

Инструкција не захтева претрагу интернета нити коришћење извора који нису прослеђени у *prompt*-у. Захтев за навођење извора формулисан је условно, изразом „када је корисно“, па модел није програмски обавезан да уз сваку тврдњу наведе формални цитат. Извори који се приказују у апликацији издвајају се из метаподатака *retrieved child* сегмената и не морају бити истоветни формалним цитатима које модел евентуално наведе у тексту одговора.

Ако је пронађен најмање један *child* резултат, *RAGPipeline* позива *Gemini* клијент. Класа *GeminiFlashClient* овај позив прослеђује *Google Gen AI SDK*-у помоћу методе *interactions.create()*.

Листинг 5.9. Позив методе *interactions.create()*

```
interaction = self.client.interactions.create(  
    model=self.model_name,  
    input=prompt,  
    system_instruction=system_instruction,  
    generation_config={  
        "max_output_tokens": self.max_output_tokens,  
    },  
    store=False,  
)
```

Табела 5. Основни параметри методе *interactions.create()*

Параметар	Значење
<i>model</i>	Назив <i>Gemini</i> модела који се користи за генерисање одговора
<i>input</i>	Главни RAG <i>prompt</i> који садржи <i>child</i> контекст, питање и задатак
<i>system_instruction</i>	Системска инструкција која одређује начин понашања модела
<i>max_output_tokens</i>	Горња граница броја токена генерисаног одговора
<i>store=False</i>	Захтев да се интеракција не чува као трајна конверзација у оквиру овог API позива

На крају методе *answer()* формира се објекат типа *RAGResponse*.

Листинг 5.10. Враћање објекта типа *RAGResponse*

```
return RAGResponse(
    question=question,
    answer=answer_text,
    sources=extract_sources(retrieved_documents),
    retrieved_documents=retrieved_documents,
    prompt=prompt,
)
```

Табела 6. Поља објекта *RAGResponse*

Поље	Садржај
<i>question</i>	Оригинално питање корисника
<i>answer</i>	Генерисани одговор или <i>fallback</i> порука ако нема резултата
<i>sources</i>	Извори издвојени из метаподатака <i>retrieved child</i> сегмената
<i>retrieved_documents</i>	<i>Child</i> документи које је вратио <i>retriever</i> , заједно са њиховим метаподацима и оценама
<i>prompt</i>	Коначни <i>prompt</i> прослеђен <i>Gemini</i> моделу

Оваква организација раздваја *retrieval child* сегмената, формирање *prompt*-а, генерацију одговора и приказ извора. Истовремено, систем детерминистички проверава само да ли је *retrieval* вратио резултате. Процена да ли је непразан контекст заиста довољан за одговор препуштена је генеративном моделу, у складу са системском инструкцијом.

5.9. Кориснички интерфејс и историја

Streamlit апликација има главни екран за постављање питања и приказ генерисаних одговора. Бочна трака омогућава избор колекције, броја коначних извора, броја почетних кандидата, ограничења броја *child* резултата по PDF документу, величине контекста и максималног броја излазних токена. Доступни прекидачи

укључују хибридную претрагу, груписање резултата и приказ *retrieved* контекста.

Испод одговора налази се табела са називом лека, активном супстанцом, PDF-ом, секцијом, страницом, типом и оценом. Прошириви приказ омогућава читање текста и сажетих метаподатака. Нема уграђеног PDF прегледача са истицањем конкретног пасуса, извоза разговора или администрације корпуса кроз UI.

Стање разговора чува се у *st.session_state*. *Streamlit* сесијско стање омогућава да подаци преживе поновно извршавање скрипте у истој сесији; не представља само по себи трајну базу разговора. [23] Нова сесија или рестарт сервера не реконструишу претходну историју из диска.

Искључивање опције за контекст мења видљивост, не брише сачуване податке. Дугме за чишћење разговора уклања све поруке и њихове доказе из сесије. Ако *backend* тренутно није доступан, стари одговори остају видљиви, а нови унос је онемогућен.

На сликама 5.3 и 5.4 приказан је стварни интерфејс са примером питања и одговора.

Podešavanja

Chroma kolekcija

diplomski_rag

Broj izvora: 5

Kandidati za pretragu: 60

Rezultati po PDF-u: 2

Konekst za Gemini: 14000

Dužina odgovora: 1200

☒ Hybrid search

☒ Grupisi po PDF-u

☒ Prikaži kontekst

Očisti razgovor

Apotekarski RAG asistent

Sistem je namenjen kao pomoc pri radu u apoteci i ne zamenjuje stručnu procenu farmaceuta ili lekara.

Šta su farmakokinetičke interakcije lekova?

Farmakokinetičke interakcije lekova se mogu javiti kod svih farmakokinetičkih procesa. Nastaju kada jedan lek utiče na resorpciju, distribuciju, metabolizam i/ili izlučivanje (ekskreciju) drugog leka.

Iako se ove interakcije mogu predvideti, zbog velike interindividualne varijabilnosti ne može se tačno predvideti u kojoj meri će biti od kliničkog značaja.

Glavni mehanizmi kroz koje se ispoljavaju su:

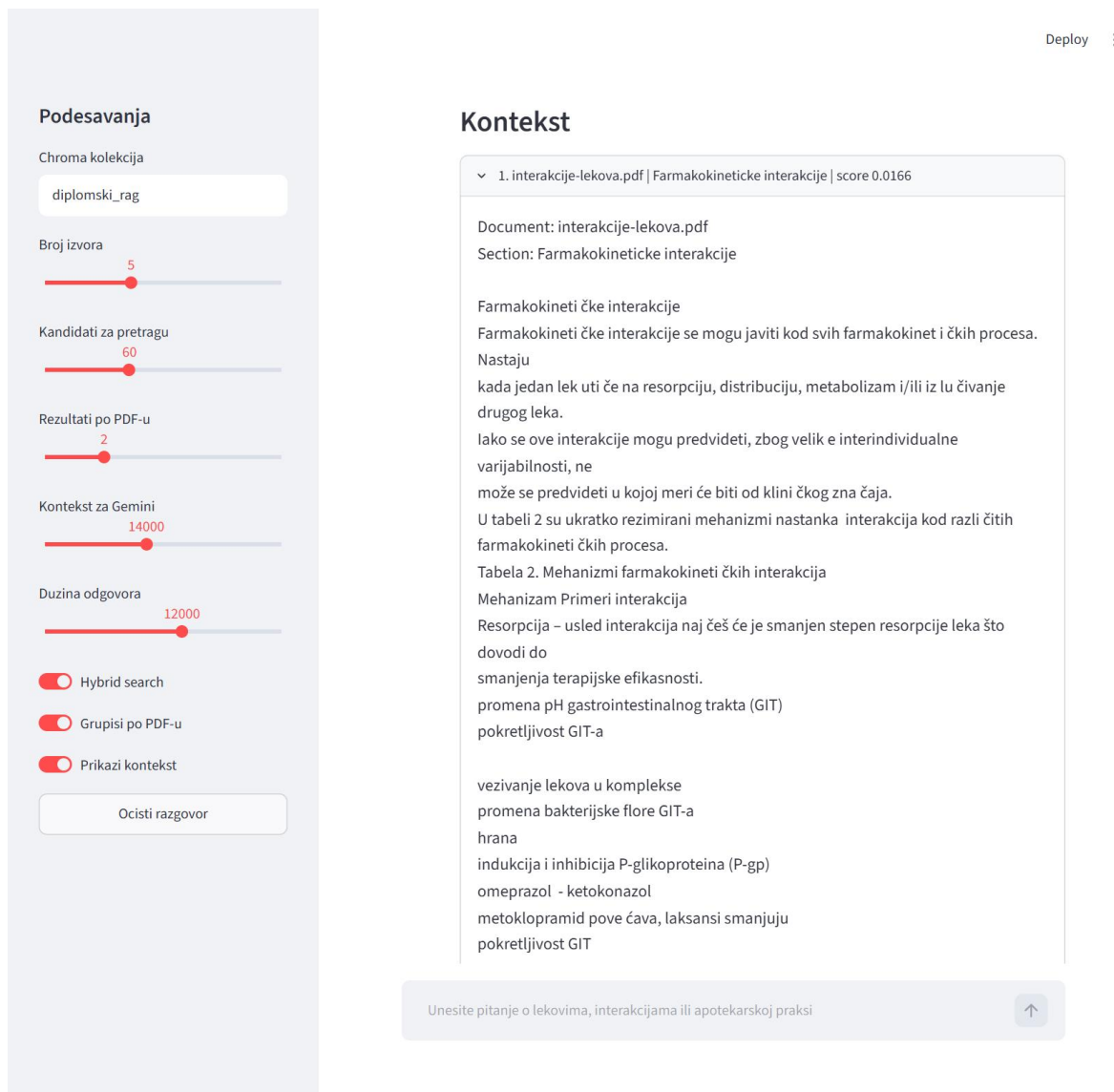
- **Resorpcija:** Najčešće dolazi do smanjenja stepena resorpcije leka (usled promena u pH vrednosti gastrointestinalnog trakta, pokretljivosti GIT-a, vezivanja lekova u komplekse, uticaja hrane, bakterijske flore ili indukcije/inhibicije P-glikoproteina).
- **Distribucija:** Dolazi do kompeticije lekova za vezivanje za proteine plazme (posebno značajno kod lekova sa visokim stepenom vezivanja za proteine plazme).
- **Metabolizam:** Predstavljaju najznačajnije interakcije, a nastaju usled indukcije ili inhibicije izoenzima CYP450.
- **Ekskrecija (izlučivanje):** Promene u bžrini glomerularne filtracije, tubularne sekrecije, tubularne reapsorpcije ili pH vrednosti i protoka urina utiču na smanjenje ili povećanje izlučivanja lekova putem bubrega.

Izvor:

- [interakcije-lekova.pdf](#), strana 6, sekcija Farmakokinetičke interakcije

Unesite pitanje o lekovima, interakcijama ili apotekarskoj praksi

Слика 5.3. Питање и одговор у Streamlit интерфејсу.



Слика 5.4. Почетак пронађене секције у интерфејсу; приказ није једнак ограниченом контексту послатом моделу.

6. Тестирање и експериментални резултати

6.1. Извори доказа и поновљивост

Ispitivanje sistema obuhvata dva nivoa dokaza. Prvi nivo čine automatizovani testovi, analiza lokalnih dokumenata i provere graničnih uslova. Oni su izvršeni 9. septembra 2026. godine, a rezultati su sačuvani u direktorijumu eval/runs/20260909T120408Z. Drugi nivo čine ranije istog dana izvršeni pozivi *Chroma Cloud* i *Gemini* servisa, sačuvani u direktorijumu eval/runs/20260909T100414Z. Novo lokalno izvršavanje nije ponovilo udaljenu pretragu, nije generisalo nove odgovore i nije menjalo kolekciju. Zato se rezultati spoljnih servisa tumače kao arhivirano merenje sa navedenim datumom, a ne kao kontinuirana garancija trenutnog ponašanja servisa. [24]

Manifest izvršavanja beleži okruženje, verzije paketa i otiske relevantnih datoteka. Osnovni evaluacioni skup je *eval/retrieval_gold.json*, dok je *eval/retrieval_challenges.json* dopunski skup zahtevnijih pitanja. Izvorni JSON rezultati zadržavaju pojedinačne upite, vraćene zapise i ocene, što omogućava da se agregati provere bez novog plaćenog poziva servisa.

Ponovljivost je delimična. Lokalni testovi nad istim kodom i PDF datotekama mogu se ponoviti, ali vreme odziva udaljenog servisa zavisi i od mreže i opterećenja. Naziv dostupnog modela nije isto što i nepromenljiv otisak njegovih težina. Za *Gemini* nisu zabeleženi svi uslovi potrebni za potpuno determinističko generisanje. Zbog toga se u radu ne tvrdi da će novo izvršavanje dati isti tekst odgovora ili identično trajanje.

6.2. Софтверски тестови

У новом локалном извршавању успешно је завршен 37 тест, за 16,209 секунди, без пријављених неуспеха. Подела по датотекама приказана је у табели 7. Део тестова користи синтетичке документе и замене зависности, док део чита стварне локалне PDF датотеке. Успешан исход потврђује проверена својства, али није доказ потпуне покривености кода или клиничке исправности одговора.

Табела 7. Расподела аутоматизованих тестова у провереној верзији

Датотека	Број	Главни предмет провере
<i>test_data_loader.py</i>	7	Типови докумената, странице, метаподаци, наставак после неисправног PDF-а
<i>test_document_sections.py</i>	7	Ћирилица, шест секција, границе секција, parent-child везе, outline водича
<i>test_retrieval_evaluation.py</i>	6	Метрике, подразумевани child-only режим и бележење изабраног режима
<i>test_retriever_rag.py</i>	4	Делегирање child претраге, child текст у prompt-у и изостанак LLM позива без контекста
<i>test_vector_store.py</i>	7	Ограничење величине, метаподаци, поновљив ID за исти улаз
<i>test_ui_history.py</i>	6	Историја одговора, извори, контекст, грешке и чишћење сесије

Тестови секција проверавају да *child* сегмент не садржи текст две *parent* секције и да се веза са *parent* секцијом задржава у метаподацима. Тест над упутством за лек проверава присуство шест главних секција.

Streamlit AppTest омогућава програмско покретање апликације и испитивање њених елемената и стања без ручног кликања у прегледачу. [25] У тестовима овог пројекта RAG зависност је замењена контролисаним одговором. Тако се изолује понашање UI-ја од доступности *Cloud* сервиса.

Додатно се проверава да промена приказа контекста не покреће поново генерисање, да стари одговор задржава сопствену копију метаподатака и да недоступност *backend*-а не скрива већ сачувану историју. Провера није визуелна оцена изгледа на различитим ширинама екрана, нити доказ трајног чувања разговора. У тестном запису постоје упозорења о застарелом *Streamlit* параметру *use_container_width*; она нису изазвала пад тестова, али представљају посао за одржавање.

6.3. Провера корпуса и сегментације

Поновљена локална анализа обухватила је 31 PDF документ и 339 страница са успешно издвојеним текстом. У корпусу је препознато 29 упутстава за лекове, један водич добре апотекарске праксе и један документ о интеракцијама лекова. Формиране су 493 логичке *parent* секције и 1.361 *child* сегмент. Код свих 29 упутстава препознато је шест главних секција и уводни део. На тај начин упутства за лекове дају 203 *parent* секције, водич 279, а документ о интеракцијама 11 *parent* секција.

Није пронађен *child* сегмент без одговарајућег *parent_id*, нити запис коме недостаје обавезни *retrieval* префикс или неки од проверених обавезних метаподатака. Ове провере потврђују структурну конзистентност локално формираних *Document* објеката пре припреме за *Cloud* упис.

Две *parent* секције премашују ограничење од 16 KiB по текстуалном *Cloud* запису. Због тога се приликом припреме за *Chroma Cloud* деле на више физичких делова. Тако се од 493 логичке *parent* секције добија 495 физичких *parent Cloud* записа, док број *child* сегмената остаје 1.361. Укупан број припремљених физичких записа зато износи 1.856.

Разлика између 493 логичке *parent* секције и 495 физичких *parent* записа не представља дуплирање секција нити додатну логичку сегментацију. Она је последица техничког ограничења величине појединачног *Cloud* записа. Физички делови задржавају исти идентификатор *parent* секције и разликују се помоћу поља *cloud_subchunk_index*.

Архивирани *Cloud* инвентар, добијен пагинираним читањем колекције, бележи исти укупан број од 1.856 физичких записа. *Parent* делови су сачувани у колекцији и векторизовани. *Parent* физички делови служе за очување структуре и праћење порекла, а не као текстуална замена за *child* контекст који се прослеђује *Gemini* моделу.

6.4. Методологија евалуације претраге

Основни скуп за евалуацију садржи 15 питања: десет питања о лековима, два питања о интеракцијама лекова и три питања о водичу добре апотекарске праксе. Очекивани резултати описани су условима над називом PDF документа, називом секције и другим метаподацима. Иако се поље у евалуационом запису назива *expected_chunks*, ознаке у овом скупу не наводе јединствене идентификаторе *child* сегмената нити обавезне доказне реченице. У мерењу враћени резултати јесу *child*

записи, али су критеријуми релевантности и даље претежно дефинисани на нивоу документа и *parent* секције. Три допунска питања додатно захтевају присуство одређеног израза у враћеном тексту, што се проверава условом *must_contain*.

Прецизност и одзив представљају стандардне мере успешности система за проналажење информација. Код ранжираних резултата мерење на првих k места усмерава пажњу на резултате које корисник или наредна компонента заиста добија. [26] У овом пројекту операционализација мера одређена је функцијама које пореде очекивања са метаподацима и текстом враћених *child* резултата. Због тога је важно нагласити да ове метрике не мере непосредно потпуност конкретног доказног пасуса, већ пре свега успешност проналажења очекиваног документа или секције.

Нека је A_q скуп очекиваних услова за питање q , а L_q^k првих највише k враћених резултата. Нека $M(r, a)$ означава да резултат r задовољава очекивање a . Тада евалуациони код рачуна:

$$P@k(q) = \frac{\sum_{r \in L_q^k} I[\exists a \in A_q: M(r, a)]}{k} \quad (4)$$

$$R@k(q) = \frac{|\{a \in A_q: \exists r \in L_q^k: M(r, a)\}|}{|A_q|} \quad (5)$$

У формули (4) именилац остаје k и када је враћено мање резултата. У формули (5) броје се различита испуњена очекивања, а не сва понављања исте релевантне секције. На пример, ако једно очекивање има један одговарајући резултат међу првих пет, добијају се вредности $P@5=0,2$ и $R@5=1$. Ко свих пет *child* резултата задовољава исто очекивање, обе мере имају вредност 1. У том случају прецизност расте због већег броја релевантних појављивања, али се број различитих покривених циљева не повећава. Зато коришћени *gold* скуп не мери непосредно разноврсност доказа нити довољност сваког *child* сегмента за формулисање одговора.

$\text{Hit}@k$ има вредност један ако постоји бар једно поклапање у првих k резултата, а нула у супротном. Реципрочни ранг за једно питање износи $1/j$, где је j позиција првог поклапања, или нула ако поклапање не постоји у првих k места. Просек тих вредности представља $\text{MRR}@k$. И остале мере су макроусредњене, па свако питање има једнаку тежину.

Наведене формуле не користе нумеричку вредност *dense* или *RRF* оцене, већ редослед резултата и испуњеност очекивања. Због тога се њима могу поредити различите врсте претраге чак и када њихови сирови скорови имају различите скале.

Поређење метаподатака нормализује велика и мала слова, дијакритичке знакове и одређене раздвајаче. Оно не представља медицински семантички суд. Ако је неки други пасус садржински релевантан, али не задовољава уписане услове, неће бити признат као тачан погодак. Обрнуто, исправан назив документа или секције може задовољити очекивање и у случају када тело *child* сегмента не садржи довољно детаља за потпун одговор.

Табела 8. Конфигурације експеримента претраге

Ознака	Рангирање	Груписање по PDF-у	Остали заједнички услови
H-G	Qwen + SPLADE, RRF	Највише два <i>child</i> резултата по PDF документу	60 кандидата, до пет резултата

Ознака	Рангирање	Груписање по PDF-у	Остали заједнички услови
H-N	Qwen + SPLADE, RRF	Искључено	Исти корпус, упити и очекивања
D-N	Qwen	Искључено	Исти корпус, упити и очекивања

Обе хибридне варијанте користе *dense* и *sparse* тежине 0,7 и 0,3, као и RRF параметар $k=60$. У H-G конфигурацији груписање ограничава број најбоље ранжираних *child* резултата на највише два по изворном PDF документу. Пошто је коришћена *child-only* варијанта, *parent* проширивање и *parent* дедупликација нису део овог мерења. Хибридна претрага ни у једном забележеном случају није прешла на резервну *dense* претрагу. Поређење H-G са H-N мери утицај груписања, а поређење H-N са D-N утицај различитог рангирања при истим осталим условима.

6.5. Резултати основног скупа

Резултати у Табели 9 показују да подразумевана варијанта претраге H-G има $\text{Hit}@1 = 0,800$, што значи да је први резултат био релевантан за 12 од 15 питања. На првих пет места најмање једно очекивање испуњено је за 13 питања. Код два питања о активној супстанци очекују се по две секције из различитих PDF докумената, због чега $\text{Recall}@1$ није једнак $\text{Hit}@1$.

Табела 9. Макроусредњени резултати над 15 питања

Варијанта	k	Precision@k	Recall@k	Hit@k	MRR@k
H-G	1	0,800	0,733	0,800	0,800
H-G	3	0,533	0,867	0,867	0,833
H-G	5	0,347	0,867	0,867	0,833
H-N	1	0,800	0,733	0,800	0,800
H-N	3	0,667	0,833	0,867	0,833
H-N	5	0,613	0,900	0,933	0,850
D-N	1	0,733	0,700	0,733	0,733
D-N	3	0,600	0,800	0,867	0,789
D-N	5	0,480	0,933	0,867	0,789

Искључивање груписања у хибридној претрази повећало је $\text{Recall}@5$ са 0,867 на 0,900, док је $\text{Hit}@5$ повећан са 13/15 на 14/15 питања. Ова разлика не означава једноставно једно додатно потпуно покривено питање, јер се очекивања не односе увек на само један документ. На пример, конфигурација H-N проналази очекивану секцију о складиштењу у водичу, али код питања о нежељеним дејствима *амлодипина* покрива само један од два очекивана PDF документа. Свих пет резултата за то питање потиче из секције о нежељеним дејствима документа *amlodin-combo.pdf*, док очекивани документ *norvasc.pdf* није пронађен. Због тога су истовремено могуће вредности $\text{Precision}@5 = 1$ и $\text{Recall}@5 = 0,5$.

У овом основном скупу H-N има веће вредности Recall@5 и MRR@5 од D-N конфигурације: 0,900 према 0,833 и 0,850 према 0,789. Ова разлика односи се на конкретна испитана питања и не представља доказ опште надмоћи хибридне претраге. H-G не покрива два очекивања која се односе на Водич, H-N не покрива питање о издавању лекова и део очекивања за амлодипин, док D-N не покрива оба наведена питања о Водичу и такође не покрива један део очекивања за амлодипин. Сам висок score појединачног child резултата не може да надокнади непокривено очекивање.

Виша вредност Precision@5 код негруписаних варијанти делом је повезана са тим што се у резултатима задржава више *child* сегмената из исте означене секције. Груписање ограничава понављање резултата из истог PDF документа, али истовремено може искључити различите пасусе истог извора који су потребни за потпун одговор. Због тога утицај груписања треба проценивати заједно са покривеношћу различитих очекивања и садржајем контекста, а не само на основу једног агрегатног показатеља.

Код H-G конфигурације одзив се не повећава између три и пет резултата. Додатна два места зато снижавају прецизност, без повећања покривености означених циљева. То не значи да је $k=3$ универзално оптимална вредност, јер скуп садржи претежно по једно очекивање по питању и није потпуно исцрпан. Ужи закључак је да повећање броја враћених *child* резултата у овом мерењу није исправило два преостала промашаја конфигурације H-G.

Раздвајање резултата по врсти документа показује неравномерност успешности. За десет питања о лековима H-G има Recall@5 = 1,000 и MRR@5 = 0,950; за два питања о интеракцијама обе мере имају вредност 1,000; док за три питања о водичу обе мере износе 0,333. Овај резултат указује на значај прилагођене обраде различитих типова докумената, али не доказује да је препознавање структуре једини узрок разлике. Групе се разликују и по формулацији питања, броју очекиваних секција и обиму докумената.

Скуп питања о лековима обухвата осам различитих PDF докумената, а не свих 29 докумената у корпусу. Формулације питања често су блиске називима стандардних секција. Зато савршен одзив H-G конфигурације на овом подскупу не треба представљати као потпуну проверу целе базе лекова, свих могућих корисничких питања или потпуности одговора о нежељеним дејствима. Проналажење једног *child* сегмента из исправне секције не значи да су пронађене све информације које се налазе у тој секцији.

6.6. Додатне језичке провере

Допунски скуп има четири питања. Проверава се да ли је одговарајућа секција пронађена, не медицинска исправност конкретног савета.

Табела 10. Дијагностички резултати допунског скупа

Питање	Оčekivana секција	H-G: погодак у првих пет	D-N: погодак у првих пет
„Која су нежељена дејства лека Норваск?“	norvasc.pdf, секција 4	Не	Да, прво место
Пропуштена доза Хапах-а, енглески упит	xanax.pdf, секција 3	Да, прво место	Да, прво место
Norvasc и грејпфру	norvasc.pdf, секција 2	Да, прво место	Да, прво место

Питање	Očekivana sekcija	H-G: погодак у првих пет	D-N: погодак у првих пет
Хапах и управљање возилом	хапах.pdf, секција 2	Да, прво место	Да, прво место

За ћирилични упит хибридна варијанта међу првим резултатима враћа секције других лекова, док густа варијанта без груписања поставља Norvasc на прво место. Ово је конкретан контрапример тврдњи да мултијезични густо модел аутоматски обезбеђује једнак квалитет целог хибридног система на оба писма. SPLADE модел је означен као енглески, али један пример није довољан за доказ да је он једини узрок грешке.

Агрегатни Hit@5 износи 0,750 за H-G и 1,000 за D-N. Пошто су истовремено промењени рангирање и груписање, ово поређење не изолује њихове појединачне утицаје. За то би требало додати H-N на истом допунском скупу и увести више латиничних, ћириличних и енглеских парафраза истог питања. Четири случаја служе за откривање проблема, не за општу оцену језичке покривености.

6.7. Провера генерисања и времена одзива

Архивирана провера целокупног RAG тока обухвата пет позива генеративном моделу *gemini-3.6-flash*. Свих пет позива вратило је непразан текстуални одговор. Табела 11 приказује посматрано понашање система и измерено трајање позива, без представљања ових примера као клинички оцењеног скупа.

Табела 11. Квалитативна провера целог RAG тока

Случај	Посматрано понашање	Трајање позива (s)
Нежељена дејства лека Norvasc	Одговор организује информације и наводи пронађени извор	29,77
Чување лека према водичу	Формиран је одговор повезан са пронађеним child контекстом	41,53
Фармакокинетичке интеракције	Појам и механизми објашњени су на основу достављеног child контекста	28,01
Измишљени назив Zorvexalin-TEST-999	Наведено је да документи не садрже тражене информације	11,47
Питање о фудбалу изван домена система	Наведено је да достављени контекст не омогућава одговор	38,45

Последња два случаја показују корисно понашање генеративног модела: модел није попунио недостатак доказа произвољним одговором. Међутим, *retriever* је и у тим случајевима вратио *child* текстове из базе. Одбијање одговора зато је настало у фази генерисања, на основу системске инструкције и процене модела, а не као резултат детерминистичке провере релевантности која би спречила позив генеративном моделу.

Ова чињеница представља ограничење система. Програм детерминистички прескаче позив *Gemini* модела само када *retrieval* не врати ниједан резултат. Ако *retrieval* врати непразну листу *child* сегмената, чак и када они нису довољно релевантни, они се прослеђују моделу, који затим процењује да ли достављени контекст подржава одговор.

Два успешна одбијања нису довољна за израчунавање поуздане стопе спречених халуцинација. Није спроведена независна оцена фармацеута, систематска провера сваке генерисане тврдње према изворном *child* сегменту нити поређење са ручно написаним референтним одговорима. Због тога се наведени резултати могу описати као функционална и квалитативна провера целокупног RAG тока, али не и као доказ безбедности медицинских препорука.

У овом раду није неопходно репродуковати дозирање или друге клиничке савете из генерисаних одговора да би се документовало софтверско понашање система. За ту сврху довољно је описати да ли је модел користио достављени *child* контекст, навео изворе и одбио одговор када контекст није подржавао постављено питање.

6.8. Анализа грешака и ваљаности закључака

Анализа је показала да део промашаја у *retrieval*-у потиче од непотпуне анотације референтног скупа, док у другим случајевима пронађени назив или секција нису праћени довољно садржајним *child* текстом. Поред тога, ограничење *prompt*-а од 14.000 знакова може довести до тога да раније рангирани *child* сегменти заузму расположиви простор и потисну касније релевантне резултате или њихове делове. Због тога се у листи извора могу појавити документи чији *child* текст није у целости ушао у коначни *prompt*, или чији је текст у *prompt*-у само делимично присутан.

Додатне провере показале су да идентификатор записа зависи од индекса уписа, па његова стабилност важи само када су сви улазни параметри и редослед обраде непромењени. Уочено је и да *chunking* у појединим случајевима може пресећи реч, што значи да подела поштује границе *parent* секција, али не гарантује увек очување границе речи или реченице унутар *child* сегмента. Новија верзија корисничког интерфејса исправно чува изворе и контекст у историји сесије, али то не представља трајну меморију нити меморију коју генеративни модел аутоматски користи у наредним питањима.

Резултате треба тумачити у оквиру ограничења коришћеног корпуса, малог броја питања, непостојања независних анотатора и могуће непотпуности *gold* скупа. У актуелној евалуацији враћају се *child* резултати, али се релевантност у великој мери оцењује на основу метаподатака о изворном документу и *parent* секцији. Због тога коришћене метрике не мере нужно да ли је пронађен најбољи доказни пасус нити да ли један *child* сегмент садржи све информације потребне за потпун одговор. Поред тога, релевантност код комбинованих лекова не треба аутоматски приписати само једној активној супстанци.

Ови налази не доводе у питање функционалност система, већ показују да је реч о провереном прототипу са мерљивим резултатима на ограниченом скупу питања. За поузданије закључке потребни су потпунији и прецизније аотирани референтни скупови, независна стручна валидација, јасно означени доказни *child* пасуси и очување тачно употребљеног контекста у коначном *prompt*-у.

7. Безбедност, ограничења и даљи развој

7.1. Стручна одговорност и коришћење извора

Намена прототипа је подршка проналажењу и тумачењу фармацеутске документације, а не самостално прописивање терапије, постављање дијагнозе или замена стручног рада фармацеута. Ова граница није само формална напомена. Она произлази из чињенице да систем може погрешно издвојити *child* сегмент, пронаћи документ који се односи на други лек, изоставити услов примене или генерисати тврдњу која није довољно поткрепљена достављеним контекстом.

Системска инструкција захтева ослањање на достављени контекст, забрањује измишљање информација и подстиче признавање недостатка доказа. Код не позива генеративни модел када *retrieval* не врати ниједан резултат. Кориснички интерфејс приказује пронађене *child* сегменте, њихове метаподатке и изворе. То су реализовани механизми који доприносе проверљивости, али ниједан од њих појединачно не доказује истинитост генерисаног одговора.

Посебно треба разликовати три својства: постојање извора, релевантност *retrieved child* сегмента и поткрепљеност конкретне тврдње тим сегментом. Документ може бити пронађен и приказан, а да не садржи одговор на постављено питање. Релевантна информација може постојати у *parent* секцији или у другом *child* сегменту, али не мора бити обухваћена изабраним резултатима. Чак и када генеративни модел добије одговарајући *child* пасус, он га може непотпуно пренети, погрешно протумачити или повезати са другом тврдњом. Поузданија верзија система морала би да проверава цео ланац — од избора сегмента до коначне тврдње — а не само присуство назива PDF документа испод одговора.

Код питања о активној супстанци посебан ризик представљају комбиновани лекови. Проналажење *child* сегмента или документа који помиње *амлодипин* не даје право да се све информације из тог документа припишу изолованом *амлодипину*. Потребно је очувати назив конкретног препарата, његов састав и околности на које се наведени текст односи. Тренутни метаподаци засновани на имену фасцикле представљају полазну организацију корпуса, а не независно проверену фармаколошку нормализацију.

Упутство за пацијента и сажетак карактеристика лека нису исти тип документа. Шест главних секција које се користе у *parser*-у односе се на структуру упутства за пацијента. При проширењу корпуса на друге регулаторне документе морали би да се уведу одговарајући типови докумената и посебна правила за препознавање њихове структуре. Погрешна класификација таквог документа могла би да доведе до примене неодговарајућих правила или до губитка структурних информација услед *fallback* обраде по страницама.

За практичну употребу, генерисани одговор треба читати заједно са релевантним приказаним *child* сегментом и проверити у оригиналном документу. Клиничке одлуке треба препустити квалификованом стручњаку. У овом раду нису доказани усаглашеност са прописима за медицински софтвер, клиничка ефикасност нити подобност система за аутономно одлучивање. Такве тврдње захтевале би другачији обим испитивања, клиничку валидацију и одговарајућу стручну процену.

7.2. Подаци, приступ и спољни сервис

Приликом индексирања, текстови PDF докумената и одабрани метаподаци напуштају локално окружење и шаљу се *Chroma Cloud* сервису. У колекцију се уписују

parent и *child* записи, заједно са њиховим текстом и метаподацима који су потребни за идентификацију извора, секције и структурне везе. Иако се обе врсте записа чувају у колекцији, примарна претрага се ограничава на *child* записе.

Приликом постављања питања текст корисничког упита користи се у удаљеној *dense*, *sparse* или хибридној претрази у оквиру *Chroma Cloud* сервиса. Након тога се питање заједно са одабраним *child* сегментима, који су ушли у ограничени *retrieval* контекст, прослеђује *Gemini* сервису. Целокупне *parent* секције се у овој варијанти не прослеђују генеративном моделу као замена за пронађене *child* сегменте. Због тога систем није у потпуности локалан, чак и када се *Streamlit* апликација покреће на рачунару корисника.

За демонстрациона питања над приложеним документима нису потребни подаци о пацијентима. У тестном окружењу не треба уносити стварне личне или здравствене податке. Пројекат не садржи механизме за анонимизацију, откривање осетљивих података, управљање пристанком или контролу приступа по кориснику. Параметар *store=False* у позиву *Interactions* API-ја сам по себи није довољан за закључак да спољни сервис не задржава никакве податке. Условe чувања и обраде података потребно је проверити у важећој документацији и подешавањима одговарајућих провајдера.

Акредитиви се учитавају из окружења, док датотека *.env.example* садржи само називе потребних променљивих. API кључеви не смеју бити укључени у текст рада, снимке екрана, примере команди или евалуационе резултате који се деле. Уколико је кључ раније објављен, његово уклањање из приказа није довољно; потребно је поништити га и издати нови код одговарајућег провајдера.

Апликација нема аутентификацију, корисничке улоге фармацеута и администратора нити раздвајање докумената по корисницима. То не представља недостатак у односу на основну намену прототипа, али ограничава његову погодност за јавно постављање и употребу у више организација. Параметри *tenant*, *database* и назив колекције одређују приступ *Chroma Cloud* ресурсима, али не представљају систем апликационих корисничких права.

Текст преузет из PDF докумената третира се као податак за претрагу и контекст, а не као програмска инструкција. Ипак, систем не садржи формалну проверу злонамерних инструкција у документима нити независну валидацију свих тврдњи које генеративни модел наведе у одговору. Чињеница да модел нема алате за измену спољних система смањује део ризика, али не спречава да злонамеран или нетачан садржај из *retrieved child* сегмента утиче на генерисани одговор. Будуће провере требало би да обухвате документе са намерно убаченим инструкцијама и да јасније раздвоје системске инструкције од текстуалних доказа преузетих из докумената.

7.3. Преостала техничка ограничења

Пројекат има неколико техничких ограничења која произилазе из начина учитавања докумената, формирања *child* сегмената, претраге и генерисања одговора.

На улазу се користи текстуални слој PDF документа. Због тога обрада зависи од тога да ли је текст у PDF-у доступан за издвајање. Скениране странице, сложене табеле и документи са више колона могу довести до непотпуно издвојеног или неправилно распоређеног текста. Правила за препознавање секција прилагођена су типовима докумената који се користе у пројекту и не представљају универзални *parser* за сва могућа упутства о лековима.

Величина текста контролише се на два различита начина. *Child* сегменти ограничавају се бројем знакова, док се записи намењени *Chroma Cloud*-у додатно

ограничавају бројем UTF-8 бајтова. Ова ограничења не представљају директну контролу броја токена које користе *embedding* модели. Пошто се *embedding* обрада извршава у *Chroma Cloud*-у, величина *child* сегмента у локалном коду сама по себи не гарантује да ће сваки запис бити обрађен без интерног скраћивања у удаљеном сервису.

У *Chroma* колекцију уписују се и *parent* и *child* записи. *Parent* записи се не дохватљају ради замене пронађених *child* резултата целокупним *parent* секцијама. Оваква организација поједностављује управљање записима, али значи да се и *parent* текстови векторизују и чувају иако нису директни кандидати у примарној *retrieval* претрази.

Поновни упис уз опцију *--reset* подразумева брисање изабране колекције и њено поновно креирање. Ако дође до прекида током новог уписа, колекција може остати само делимично попуњена. Поступак зато не представља атомску замену старе верзије базе новом верзијом. Поред тога, стабилност идентификатора зависи од вредности које се користе при њиховом формирању, укључујући индекс записа, па промена улазних параметара може довести до различитих идентификатора.

Бројање записа са одређеним метаподацима врши се посебним читањем колекције. Такав резултат треба тумачити у складу са конкретним условима тог позива, а не аутоматски као потпун и независно пагиниран инвентар свих записа у бази. Општи број записа колекције и број записа добијен применом одређеног филтера немају нужно исти обухват.

Retrieval компонента нема посебан модул за препознавање назива лека у питању, контролисани речник синонима, специјализовани филтер по препарату или *cross-encoder* за додатно рерангирање. Релевантност се одређује *dense* и *sparse* претрагом, уз могуће RRF комбиновање резултата. Груписање по изворном PDF документу ограничава број *child* резултата из једног извора, али не гарантује да ће сваки враћени резултат припадати различитој *parent* секцији. Због тога коначна листа може садржати мање различитих секција од очекиваног броја.

Прелазак са хибридне на *dense* претрагу у случају грешке повећава отпорност система на проблеме у хибридној путањи. Ипак, пошто се у тој грани обухвата шири тип изузетка, стварни узрок проблема може остати неразјашњен ако се исписано упозорење додатно не анализира.

У генеративном делу система постоји ограничење у начину избора текста који улази у *prompt*. Контекст се попуњава редоследом *retrieval child* резултата и ограничава на задати број знакова. Ако су раније ранжирани *child* сегменти дугачки, они могу заузети расположиви простор и потиснути касније резултате или њихове делове. Систем зато не гарантује да ће сви релевантни *child* сегменти бити укључени у коначни *prompt*, чак и када се налазе у истој *parent* секцији или изворном документу.

Апликација независно формира контекст за *prompt* и листу извора за приказ. Због тога извор може бити приказан у апликацији иако је текст његовог *child* сегмента делимично или у потпуности изостављен из контекста прослеђеног *Gemini* моделу. Такође, не постоји аутоматска програмска провера којом би се потврдило да свака тврдња у генерисаном одговору има одговарајућу референцу у достављеном контексту.

Историја разговора у корисничком интерфејсу служи за приказ претходних питања, одговора, извора и контекста. Она не представља трајну меморију која се користи за аутоматско проширивање наредног питања. Због тога питања која зависе од претходног одговора морају бити формулисана тако да садрже довољно самосталног контекста.

Наведена ограничења не спречавају коришћење система као прототипа за претрагу и генерисање одговора на основу фармацеутских докумената. Она, међутим, ограничавају степен аутоматизације, контроле извора, преносивости и поузданости који се може очекивати без додатне обраде докумената, напреднијег рерангирања,

прецизнијег управљања контекстом и формалне провере генерисаних тврдњи.

7.4. Приоритети за унапређење

Први приоритет је потпуност доказног контекста. Непосредно прослеђивање текста изабраних *child* сегмената већ је реализовано у актуелној варијанти система; наредни корак није његово поновно увођење, већ испитивање контролисаног додавања суседних пасуса када изабрани *child* сегмент не садржи све услове потребне за одговор. При томе би пронађени *child* пасус морао да остане очуван као примарни доказ, уз контролу дужине контекста и преклапања информација. Такво решење представљало би селективно проширивање контекста, а не аутоматску замену *child* сегмента целокупном *parent* секцијом. Из истог коначног списка блокова који се стварно прослеђују моделу требало би формирати и изворе и приказ у корисничком интерфејсу. Нови тестови требало би да провере очуваност конкретног доказа, потпуност услова и број извора у условима ограниченог контекста.

Други приоритет је исправка граница *child* сегмената. Преклапање треба бирати на граници речи или реченице, без произвољног одсецања завршетка. За веома дугачке листе и табеле потребан је посебан поступак који очувава наслов, колоне, јединице и услове. Пре замене целог *parser*-а било би рационално формирати визуелно проверен подскуп проблематичних PDF страница и измерити да ли ново решење заиста боље очува њихов садржај.

Трећи приоритет је проширење и ручна ревизија референтних очекивања. *Gold* скуп требало би да разликује исправан документ, исправну секцију и конкретан доказни пасус. За свако питање потребно је навести све прихватљиве алтернативе у корпусу и образложити њихову релевантност. Скуп би требало да обухвати ћириличке и латиничке парове, парафразе, двосмислене називе, питања о више лекова и питања без одговора. Фармацеут би требало независно да провери очекивања и генерисане тврдње. Тек такав скуп омогућава поузданије поређење различитих конфигурација.

Четврти приоритет је побољшање претраге уз контролисане експерименте. На истим питањима, над истом верзијом корпуса и уз исту конфигурацију индекса, требало би упоредити густу и хибридную претрагу, различите тежине, правила избора RRF кандидата, груписање по PDF документу или *parent* секцији и евентуални *reranker*. Посебно треба мерити понављање *child* сегмената из исте *parent* секције и покривеност различитих доказних пасуса. Речник назива препарата и супстанци треба засновати на провереним подацима. Филтер по леку не сме аутоматски искључити документе о интеракцијама или поступцима када су они потребни за питање. Ово су предлози за наредну верзију, а не опис измена које су већ примењене у новом мерењу.

Пети приоритет је репродуктивна и недеструктивна изградња базе. Идентификатор сегмента требало би извести из верзије изворног документа, логичке секције и локалног индекса, без ослањања на глобални редни број уписа. Нова верзија корпуса могла би да се гради у засебној колекцији и активира тек након провере. Манифест би бележио отисак PDF документа, верзију *parser*-а, параметре сегментације и конфигурацију *embedding* и генеративних модела. Прекид уписа и наставак рада требало би да буду обухваћени интеграционим тестом.

Шести приоритет је употребљивост и надзор. Приказ оригиналног PDF документа са бројем странице и означеним доказним пасусом олакшао би стручну проверу. Структурирани записи о времену *retrieval*-а, броју *child* сегмената послатих моделу, скраћивању контекста и грешкама омогућили би откривање регресија. Трајна историја, ако буде потребна, морала би да има јасно дефинисану политику приватности и приступа. Наведене ставке представљају предложене могућности за унапређење, а не

функционалности које су већ доказане овим радом.

8. Закључак

У раду је реализован и анализиран RAG прототип за подршку раду у апотеци. Теоријске основе проналажења информација, мултијезичних векторских репрезентација и генерисања повезане су са конкретним током обраде PDF документације. *LangChain Document* објекти представљају заједнички формат текста и метаподатака, *Chroma Cloud* обезбеђује складиштење и густу и хибридную претрагу, а *Gemini* формира одговор из питања и изабраног контекста.

Главни практични допринос је раздвајање читавања, препознавања структуре, *parent-child* сегментације, претраге и генерисања. Упутства за лекове обрађују се према шест главних секција, водич према доступној *outline* структури, а документ о интеракцијама посебним правилима. Актуелни RAG прослеђује пронађене *child* текстове без замене целим *parent* секцијама; везе и префикси чувају порекло. Интерфејс омогућава преглед одговора и доказа и задржавање тих података у истој сесији.

Поновљена анализа обухватила је 31 PDF, 339 страница, 493 логичке *parent* секције и 1.361 *child* сегмената. Свих 37 аутоматизованих тестова је прошло. Над постојећом *Cloud* базом изведене су 53 евалуационе претраге и пет стварних RAG провера, без измене колекције. На основном скупу H-G има *Precision@5* 0,347 и *Recall@5* 0,867, H-N 0,613 и 0,900, а D-N 0,480 и 0,833. На четири захтевнија питања *Hit@5* износи 0,250 за H-G и 0,750 за D-N. Зато основни и допунски резултати не подржавају општу тврдњу да је једна стратегија увек боља.

Проверени *child* текстови очувани су у форматираном контексту и сачуваним RAG *prompt*-овима, али потпун пренос пронађеног текста није исто што и потпун одговор. Више деце исте секције може повећати прецизност без нових доказа, док избор преуског пасуса може изоставити потребан услов. Преостали приоритети су ознаке конкретних доказних пасуса, прецизније границе сегмената, контрола понављања и потпуности контекста, независна стручна евалуација и репродуктивније управљање базом. Реализовани систем представља проверљиву основу за даљи развој подршке фармацеуту, не клинички валидирану замену за његову процену.

Литература

- [1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel и D. Kiela, „Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks“, *Advances in Neural Information Processing Systems*, 2020. Досупно на: <https://arxiv.org/abs/2005.11401>.
- [2] Пројектни репозиторијум дипломски, „Изворни код система за подршку раду у апотеци“, верзија пакета 0.1.0. Изворни код (../src/diplomski/).
- [3] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser и I. Polosukhin, „Attention Is All You Need“, *Advances in Neural Information Processing Systems*, 2017. Досупно на: <https://arxiv.org/abs/1706.03762>.
- [4] N. Reimers и I. Gurevych, „Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks“, *Proceedings of EMNLP-IJCNLP*, 2019. Досупно на: <https://arxiv.org/abs/1908.10084>.
- [5] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen и W.-t. Yih, „Dense Passage Retrieval for Open-Domain Question Answering“, *Proceedings of EMNLP*, 2020. Досупно на: <https://arxiv.org/abs/2004.04906>.
- [6] T. Formal, B. Piwowarski и S. Clinchant, „SPLADE: Sparse Lexical and Expansion Model for First Stage Ranking“, *Proceedings of SIGIR*, 2021. Досупно на: <https://arxiv.org/abs/2107.05720>.
- [7] G. V. Cormack, C. L. A. Clarke и S. Büttcher, „Reciprocal Rank Fusion outperforms Condorcet and individual Rank Learning Methods“, *Proceedings of SIGIR*, 2009, doi: 10.1145/1571941.1572114. Досупно на: <https://cormack.uwaterloo.ca/cormacksigir09-rrf.pdf>.
- [8] Qwen, „Qwen3-Embedding-0.6B: Model Card“, Hugging Face. Досупно на: <https://huggingface.co/Qwen/Qwen3-Embedding-0.6B>.
- [9] Chroma, „Chroma Cloud Qwen“, документација интеграције векторизације. Досупно на: <https://docs.trychroma.com/integrations/embedding-models/chroma-cloud-qwen>.
- [10] Chroma, „Chroma Cloud Splade“, документација интеграције ретких репрезентација. Досупно на: <https://docs.trychroma.com/integrations/embedding-models/chroma-cloud-splade>.
- [11] prithivida, „Splade_PP_en_v1: Model Card“, Hugging Face. Досупно на: https://huggingface.co/prithivida/Splade_PP_en_v1.
- [12] Chroma, „Manage Collections“, документација. Досупно на: <https://docs.trychroma.com/docs/collections/manage-collections>.
- [13] Chroma, „Sparse Vector Search“, документација Cloud шеме. Досупно на: <https://docs.trychroma.com/cloud/schema/sparse-vector-search>.
- [14] Chroma, „Hybrid Search“, документација Search API-ја. Досупно на: <https://docs.trychroma.com/cloud/search-api/hybrid-search>.
- [15] Chroma, „Group By“, документација Search API-ја. Досупно на: <https://docs.trychroma.com/cloud/search-api/group-by>.
- [16] Chroma, „Chunking“, водич за изградњу система. Досупно на: <https://docs.trychroma.com/guides/build/chunking>.

- [17] Google, „Gemini 3.6 Flash“, документација модела Gemini API. Досупно на: <https://ai.google.dev/gemini-api/docs/models/gemini-3.6-flash>.
- [18] Google, „Interactions API Overview“, Gemini API документација. Досупно на: <https://ai.google.dev/gemini-api/docs/interactions-overview>.
- [19] European Medicines Agency, „QRD Product-information Annotated Template, English, Version 10.4“. Досупно на: https://www.ema.europa.eu/en/documents/template-form/qrd-product-information-annotated-template-english-version-104_en.pdf.
- [20.1] Агенција за лекове и медицинска средства Србије (АЛИМС), „Pretraživanje humanih lekova – PIS“. Досупно на: <https://www.alims.gov.rs/humanilekovi/pretrazivanje-humanih-lekova/?a=p>.
- [20.2] Министарство здравља Републике Србије, водич добре апотекарске праксе, 2021. Досупно на: https://www.zdravlje.gov.rs/view_file.php?cache=sr&file_id=2168.
- [20.2] Фармацеутска комора Србије, интеракције лекова. Досупно на: <https://www.farmkom.rs/pdf/stranice/interakcije-lekova.pdf>.
- [21] LangChain, „Document Loaders“, документација Python интеграција. Досупно на: https://docs.langchain.com/oss/python/integrations/document_loaders.
- [22] pypdf, „Extract Text from a PDF“, документација. Досупно на: <https://pypdf.readthedocs.io/en/stable/user/extract-text.html>.
- [23] Streamlit, „Session State“, документација API-ја. Досупно на: https://docs.streamlit.io/develop/api-reference/caching-and-state/st.session_state.
- [24] Projektni repozitorijum diplomski, „Cloud inventar, poređenje pretrage i RAG proverе“, извршавање 20260909T100414Z, 09.09.2026. Artefakti eksperimenta (../eval/runs/20260909T100414Z/).
- [25] Streamlit, „st.testing.v1.AppTest“, документација тестирања апликација. Досупно на: <https://docs.streamlit.io/develop/api-reference/app-testing/st.testing.v1.apptest>.
- [26] C. D. Manning, P. Raghavan и H. Schütze, *Introduction to Information Retrieval*. Cambridge University Press, 2008, одељак „Evaluation of ranked retrieval results“. Досупно на: <https://nlp.stanford.edu/IR-book/html/htmledition/evaluation-of-ranked-retrieval-results-1.html>

